

Log Management System Documentation: Data Structures

Medjitna Brlkacem/Lahmar Meriem Imene

May 18, 2025

Contents

1	System Overview	4
1.1	Purpose and Scope	4
1.2	Key Components and Data Structures	4
1.3	Core Functionalities	4
1.4	Algorithmic Techniques and Analysis	5
1.5	Other Essential Aspects	5
2	Introduction to Linked Lists	5
3	Log Entry Structure (Singly Linked List)	5
4	Singly Linked List Implementation Details	6
4.1	Creating a New Log Entry Node	6
4.2	Checking if the List is Empty	7
4.3	Adding a Log Entry at the Beginning	7
4.4	Adding a Log Entry at the End	7
4.5	Inserting a Log Entry at a Specific Position	8
4.6	Displaying All Log Entries	9
4.7	Getting the Number of Log Entries	9
4.8	Deleting a Log Entry by ID	10
4.9	Deleting the First Log Entry	10
4.10	Deleting the Last Log Entry	11
4.11	Deleting a Log Entry by Timestamp	11
4.12	Sorting Logs by Severity (Distribution Sort)	13
4.13	Reversing the Linked List	14
4.14	Searching for a Log Entry by ID	14
4.15	Generating Multiple Log Entries	15
5	Introduction	15
6	Log Entry Structure (Bidirectional Linked List)	15
7	Bidirectional Linked List Implementation Details	16
7.1	Creating a New Bidirectional Log Entry Node	16
7.2	Checking if the Bidirectional List is Empty	17
7.3	Adding a Log Entry at the Beginning (Bidirectional)	17
7.4	Adding a Log Entry at the End (Bidirectional)	17
7.5	Inserting a Log Entry at a Specific Position (Bidirectional)	18
7.6	Displaying All Log Entries (Bidirectional)	19
7.7	Getting the Number of Log Entries (Bidirectional)	20
7.8	Deleting a Log Entry by ID (Bidirectional)	21
7.9	Deleting the First Log Entry (Bidirectional)	21
7.10	Deleting the Last Log Entry (Bidirectional)	22
7.11	Deleting a Log Entry by Position (Bidirectional)	22
7.12	Deleting a Log Entry by Timestamp (Bidirectional)	23

8	Memory Management	25
8.1	Log Creation and Allocation	25
8.2	Log Deletion and Deallocation	25
8.3	Handling Memory Errors	25
9	Sorting Algorithms	25
9.1	Severity-Based Sorting	25
9.2	Merge Sort Implementation (Bidirectional)	26
9.3	Other Sorting Criteria	26
10	Error Handling	26
10.1	Timestamp Generation Errors	26
10.2	Memory Allocation Errors	26
10.3	Invalid User Input	26
11	Search Operations	26
11.1	ID-Based Search	26
11.2	Timestamp-Based Search	26
11.3	Message Content Search	27
12	Deletion Methods	27
12.1	Deleting by ID	27
12.2	Deleting the Last Entry	27
12.3	Timestamp-Based Deletion	28
12.4	Deleting by Position (Bidirectional)	28
13	Utility Functions	28
13.1	List Reversal	28
13.2	Getting List Size	28
13.3	Checking if List is Empty	29
13.4	Generating Logs	29
13.5	Navigating Logs (Bidirectional)	29
14	Algorithmic Complexity Analysis	29
14.1	Understanding Big-O Notation	29
14.2	Singly Linked List Operation Complexities	29
14.3	Bidirectional Linked List Operation Complexities	30
14.4	Linked List Advantages	31
14.5	Merge Sort Complexity	31
15	Testing Methodology	31
15.1	Unit Test Cases	31
16	Performance Benchmarks	32
17	Introduction	32
18	Log Entry Structure (Circular Linked List)	32
19	Circular Linked List Implementation Details	33
19.1	Creating a New Log Entry Node	33
19.2	Checking if the Circular List is Empty	33
19.3	Adding a Log Entry at the Beginning (Circular)	34
19.4	Displaying All Log Entries (Circular)	36
19.5	Sorting Logs by ID (Circular - Merge Sort)	41
19.6	Sorting Logs by Severity (Circular)	42
19.7	Reversing the Circular Linked List	43
19.8	Searching for a Log Entry by ID (Circular)	44
19.9	Generating Multiple Log Entries (Circular)	44

20 Algorithmic Complexity Analysis (Circular Linked Lists)	45
21 Introduction	46
22 Log Entry Structure (Queue)	46
23 Queue Structure	46
24 Queue Implementation Details	47
24.1 Initializing the Queue	47
24.2 Enqueuing a Log Entry	47
24.3 Dequeuing a Log Entry	48
24.4 Peeking at the Front Entry	48
24.5 Checking if the Queue is Empty	48
24.6 Checking if the Queue is Full	49
25 Algorithmic Complexity Analysis (Queues)	49
26 Introduction	50
27 Log Entry Structure (Stack)	50
28 Stack Structure	50
29 Stack Implementation Details	50
29.1 Initializing the Stack	50
29.2 Checking if the Stack is Empty	51
29.3 Checking if the Stack is Full	51
29.4 Pushing a Log Entry onto the Stack	51
29.5 Popping a Log Entry from the Stack	52
29.6 Peeking at the Top Entry	52
29.7 Checking Stack State	52
29.8 Inserting at the Bottom (Recursive Helper)	52
29.9 Reversing the Stack	53
29.10 Displaying a Single Log Entry	53
30 Algorithmic Complexity Analysis (Stacks)	53
31 Comparison of Queues and Stacks	54
31.1 Fundamental Principles	54
31.2 Core Operations	54
31.3 Typical Use Cases	54
31.4 Summary Table	55
32 Introduction to Recursion	55
33 Recursive Functions in the Log Management System	55
33.1 Factorial Calculation	55
33.2 Fibonacci Series Calculation	56
33.3 Recursive Linked List Reversal	56
33.4 Finding Maximum Log ID Recursively	57
34 Algorithmic Complexity Analysis (Recursion)	57
35 Introduction to Trees	58
36 Introduction to Binary Search Trees	58
37 Log Entry Structure (Binary Search Tree)	59

38 Binary Search Tree Implementation Details	59
38.1 Creating a New BST Log Entry Node	59
38.2 Adding a Node to the BST (Recursive Helper)	60
38.3 Adding a Log Entry to the BST	61
38.4 Displaying a Single BST Log Entry Node	61
38.5 Finding the Minimum Node in a Subtree	61
38.6 Deleting a Node from the BST (Recursive)	62
38.7 Deleting a Node from the BST	63
38.8 BST Traversal: In-Order	63
38.9 BST Traversal: Pre-Order	63
38.10 BST Traversal: Post-Order	64
38.11 Converting a Linked List to a BST	65
38.12 Queue Implementation for Level Order Traversal (Helper)	65
38.13 Finding and Deleting the Minimum Node (Alternative/Helper)	66
39 Algorithmic Complexity Analysis (Binary Search Trees)	67
40 Conclusion	68

1 System Overview

This section provides a high-level overview of the Log Management System, its purpose, and key components.

1.1 Purpose and Scope

This document provides comprehensive documentation for a Log Management System implemented in C. The system is designed to capture, store, manage, and process log entries, offering various functionalities for handling log data efficiently using different data structures and algorithmic techniques. The core purpose of the system is to demonstrate and compare the application of fundamental data structures and algorithms in a practical context.

1.2 Key Components and Data Structures

The system allows users to interact with log data stored in several fundamental data structures:

- **Linked Lists:** Including Singly Linked Lists, Bidirectional Linked Lists, and Circular Linked Lists. These dynamic structures are explored for their flexibility in handling varying numbers of log entries and their performance characteristics for operations like insertion, deletion, and traversal.
- **Queues:** An array-based implementation of a Queue is included, demonstrating First-In, First-Out (FIFO) data management, which could be used for processing logs in the order they are received.
- **Stacks:** An array-based implementation of a Stack is included, demonstrating Last-In, First-Out (LIFO) data management, potentially useful for tasks like undo operations or processing logs in reverse order.
- **Binary Search Trees (BSTs):** A BST implementation is used to store log entries, ordered by timestamp. This structure facilitates efficient searching, insertion, and deletion operations, particularly useful for quickly retrieving logs within a specific time range.

1.3 Core Functionalities

The system supports a range of operations on these data structures, including:

- Adding new log entries (at the beginning, end, or specific positions).
- Deleting log entries (by ID, position, or timestamp).
- Searching for log entries (by ID or keyword).

- Sorting log entries (by severity or ID).
- Displaying log entries.
- Calculating the number of log entries.
- Converting data between different structures (e.g., Linked List to BST).

1.4 Algorithmic Techniques and Analysis

Additionally, the documentation delves into **Recursion** as an algorithmic technique, showcasing its application in mathematical calculations (factorial, Fibonacci) and linked list manipulation (reversal, finding maximum ID). A significant part of this documentation is dedicated to the **Algorithmic Complexity Analysis** of the implemented operations across all data structures. This analysis, using Big-O notation, evaluates the time and space efficiency of each function, providing insights into how the performance scales with the size of the log data. Understanding these complexities is vital for choosing the most appropriate data structure and algorithm for specific log management tasks and for predicting system performance under different loads.

1.5 Other Essential Aspects

The document also covers essential aspects like **Memory Management**, ensuring efficient allocation and deallocation of memory to prevent leaks; **Error Handling**, outlining mechanisms to deal with potential issues like memory allocation failures or invalid user input; and the **Testing Methodology** used to ensure the system's reliability and correctness.

In summary, this Log Management System serves as a practical exploration of fundamental data structures and algorithms, providing a foundation for understanding their implementation, application, and performance characteristics in the context of managing log data.

Singly Linked Lists

2 Introduction to Linked Lists

Linked lists are fundamental linear data structures where elements are not stored at contiguous memory locations. Instead, each element is a separate object, called a node, that contains a data part and a pointer to the next node in the sequence. This structure allows for efficient insertion and deletion of elements compared to arrays, where these operations can be costly as they may require shifting elements.

Singly linked lists are the simplest type of linked list, where each node contains a data part and a pointer to the next node.

In the context of a log management system, singly linked lists are a suitable choice for storing log entries due to their dynamic size and efficient insertion/deletion at the ends.

3 Log Entry Structure (Singly Linked List)

The basic building block for storing log information in a singly linked list is the `LogEntry` structure.

```

1 struct LogEntry {
2     int id; // Auto-incremented identifier
3     char message[100]; // Log content
4     struct LogEntry *next; // Pointer to next entry
5     char severity[10]; // INFO/WARNING/ERROR
6     char timestamp[50]; // Formatted timestamp
7 };
8 typedef struct LogEntry LogEntry;
```

Structure Breakdown:

‘id’: An integer to uniquely identify each log entry. ‘message’: A character array to store the log message content. A fixed size of 100 is used. ‘next’: A pointer of type ‘struct LogEntry *’ that points to the subsequent log entry in the list. ‘severity’: A character array to store the severity level of the log (e.g., “INFO”, “WARNING”, “ERROR”). A fixed size of 10 is used. ‘timestamp’: A character array to store the timestamp of when the log was created, formatted as a string. A fixed size of 50 is used.

The ‘typedef’ statement creates an alias ‘LogEntry’ for ‘struct LogEntry’.

4 Singly Linked List Implementation Details

4.1 Creating a New Log Entry Node

The CreateLogEntry function allocates memory for a new LogEntry node and initializes its fields.

```
1 #include <stdlib.h> // For malloc and free
2 #include <stdio.h> // For printf, fgets, scanf
3 #include <string.h> // For strncpy, strlen, strcmp, strstr
4 #include <stdbool.h> // For bool type
5 #include <time.h> // For time functions (assuming getCurrentTimeString uses these)
6
7 // Assume ID is a global variable defined elsewhere, e.g., int ID = 0;
8 extern int ID;
9 // Assume getCurrentTimeString() is defined elsewhere and returns a dynamically
10 // allocated string.
11 extern char* getCurrentTimeString();
12 // Assume isValidDateTime is defined elsewhere
13 extern bool isValidDateTime(int year, int month, int day, int hour, int minutes, int
14 // seconds);
15
16 LogEntry* CreateLogEntry(){
17     LogEntry* temp = (LogEntry*)malloc(sizeof(LogEntry));
18     if(temp == NULL) {
19         printf("\nMemory Allocation Failed\n");
20         return NULL;
21     }
22
23     temp->id = ID++; // Assuming ID is a global counter
24     temp->next = NULL;
25     char msg[100];
26     char msg2[100]; // Used for message input
27     char* temp3 = getCurrentTimeString(); // Function to get timestamp
28
29     if (temp3) {
30         strncpy(temp->timestamp, temp3, sizeof(temp->timestamp) - 1);
31         temp->timestamp[sizeof(temp->timestamp) - 1] = '\0'; // Ensure null-
32         termination
33         free(temp3); // Free memory allocated by getCurrentTimeString
34     } else {
35         strncpy(temp->timestamp, "ERROR", sizeof(temp->timestamp) - 1);
36         temp->timestamp[sizeof(temp->timestamp) - 1] = '\0';
37     }
38
39     printf("\nWhat is the message?\n");
40     fgets(msg2, sizeof(msg2), stdin);
41     size_t len = strlen(msg2);
42     if (len > 0 && msg2[len - 1] == '\n') {
43         msg2[len - 1] = '\0'; // Remove trailing newline
44     }
45     strncpy(temp->message, msg2, sizeof(temp->message) - 1);
46     temp->message[sizeof(temp->message) - 1] = '\0';
47
48     do {
49         printf("What is the level of severity (INFO, WARNING, ERROR)?\n");
50         if (fgets(msg, sizeof(msg), stdin) != NULL) {
51             size_t len = strlen(msg);
52             if (len > 0 && msg[len - 1] == '\n') {
53                 msg[len - 1] = '\0';
54             }
55             strncpy(temp->severity, msg, sizeof(temp->severity) - 1);
56             temp->severity[sizeof(temp->severity) - 1] = '\0';
57         } else {
58             printf("Error reading severity level.\n");
59             free(temp); // Free allocated memory on input error
60             return NULL;
61         }
62     } while (strcmp(temp->severity, "INFO") != 0 && strcmp(temp->severity, "WARNING")
63 != 0 && strcmp(temp->severity, "ERROR") != 0);
```

```

61
62     return temp;
63 }

```

Functionality:

Allocates memory for a new `LogEntry` using `malloc`. Increments a global ID counter and assigns it to the new entry's `id`. Sets the `next` pointer to `NULL`. Calls `getCurrentTimeString` to get the current timestamp string and copies it to the `timestamp` field using `strncpy` for safety. Prompts the user for the log message and severity level, reading input using `fgets` and handling potential newline characters. Validates the entered severity level to be one of "INFO", "WARNING", or "ERROR". Returns a pointer to the newly created `LogEntry` node, or `NULL` if memory allocation or input failed.

4.2 Checking if the List is Empty

The `IsLogsEmpty` function provides a simple check to see if the singly linked list is currently empty.

```

1 bool IsLogsEmpty(LogEntry*L){
2     return (L==NULL);
3 }

```

Functionality:

Takes the head pointer of the list (`L`) as input. Returns `true` if the head pointer is `NULL` (indicating an empty list), and `false` otherwise.

4.3 Adding a Log Entry at the Beginning

The `AddLogEntryAtTheBeginning` function inserts a new log entry at the front of the singly linked list.

```

1 void AddLogEntryAtTheBeginning(LogEntry**L){
2     bool v=IsLogsEmpty(*L);
3     LogEntry*temp=CreateLogEntry();
4
5     if (temp==NULL)
6     {
7         printf("Allocation of Mem failed\n");
8         return;
9     }
10
11     if (v)
12     {
13         (*L)=temp; // New node becomes the head
14     }
15     else
16     {
17         temp->next=(*L); // New node's next points to the current head
18         (*L)=temp; // New node becomes the new head
19     }
20 }

```

Functionality:

Calls `CreateLogEntry` to get a new log entry node. If the list is empty, the new node becomes the head. If the list is not empty, the new node's `next` pointer is set to the current head, and the list's head pointer is updated to point to the new node.

4.4 Adding a Log Entry at the End

The `AddLogEntryAtTheEnd` function appends a new log entry to the end of the singly linked list.

```

1 void AddLogEntryAtTheEnd(LogEntry**L){
2     bool v=IsLogsEmpty(*L);
3     LogEntry*temp=CreateLogEntry();
4
5     if (temp==NULL)
6     {
7         printf("Addition Failed\n"); // More specific error message

```

```

8         return;
9     }
10
11     if (v)
12     {
13         (*L)=temp; // New node becomes the head if list is empty
14     }
15     else
16     {
17         LogEntry*Tem=(*L);
18         while (Tem->next!=NULL)
19         {
20             Tem=Tem->next; // Traverse to the last node
21         }
22         Tem->next=temp; // Last node's next points to the new node
23     }
24 }

```

Functionality:

Calls `CreateLogEntry` to get a new log entry node. If the list is empty, the new node becomes the head. If the list is not empty, it traverses the list to find the last node (the one whose `next` pointer is `NULL`), and then sets the last node's `next` pointer to the new node.

4.5 Inserting a Log Entry at a Specific Position

The `InsertLogEntry` function inserts a new log entry at a specified position within the singly linked list. `InsertLogEntryByPos` is a helper to get the position from the user.

```

1 void InsertLogEntry(LogEntry** L, int pos) {
2     int counter = 1;
3     // Create the new log entry using the dedicated function
4     LogEntry* temp = CreateLogEntry();
5     if (temp == NULL) {
6         printf("\nFailed to create new log entry.\n");
7         return;
8     }
9
10    if (pos == 1 || *L == NULL) {
11        temp->next = *L;
12        *L = temp; // Insert at the beginning
13        return;
14    }
15
16    LogEntry* current = *L;
17    LogEntry* previous = NULL;
18
19    // Traverse to the node before the desired position
20    while (current != NULL && counter < pos) {
21        previous = current;
22        current = current->next;
23        counter++;
24    }
25
26    if (counter == pos) {
27        // Insert at the specified position
28        temp->next = current;
29        if (previous != NULL) {
30            previous->next = temp;
31        } else {
32            // This case should ideally be covered by pos == 1, but as a safeguard
33            :
34            *L = temp;
35        }
36    } else {
37        // Position is out of range (beyond the end of the list)
38        printf("The position is out of range.\n");
39        free(temp); // Free allocated memory if position is invalid
40    }
41 }
42 void InsertLogEntryByPos(LogEntry** L) {

```



```

43     int pos;
44     printf("Enter the position: ");
45     scanf("%d", &pos);
46     getchar(); // Consume leftover '\n'
47     InsertLogEntry(L, pos);
48 }

```

Functionality:

Takes the head pointer of the list and the desired insertion position as input. Handles insertion at the beginning as a special case. For other positions, it traverses the list to find the node *before* the insertion point. Updates the **next** pointers of the previous node and the new node to insert the new node into the list. If the specified position is beyond the end of the list, it prints an error message and frees the allocated memory for the new node.

4.6 Displaying All Log Entries

The DisplayLogs function iterates through the singly linked list and prints the details of each log entry.

```

1 void displayLogHeader() {
2     printf("%-5s | %-32s | %-10s | %-100s\n",
3           "ID", "Timestamp", "Severity", "Message");
4     printf("
5     -----|-----|-----|-----
6     n");
7 }
8
9 void displayLogEntryColumn(LogEntry *entry) {
10    printf("%-5d | %-32s | %-10s | %-100s\n",
11          entry->id, entry->timestamp, entry->severity, entry->message);
12 }
13
14 void DisplayLogs(LogEntry* L) {
15     if (L == NULL) {
16         printf("Logs are empty\n");
17         return;
18     }
19     displayLogHeader(); // Print header before displaying logs
20     while (L != NULL) {
21         displayLogEntryColumn(L); // Display details of current log entry
22         L = L->next; // Move to the next log entry
23     }
24 }

```

Functionality:

Takes the head pointer of the list as input. Checks if the list is empty and prints a message if it is. If the list is not empty, it prints a header for the log entries. It then traverses the list from the head, printing the formatted details of each LogEntry using displayLogEntryColumn until the end of the list is reached (NULL).

4.7 Getting the Number of Log Entries

The LogSize function calculates and returns the total number of log entries in the singly linked list.

```

1 int LogSize(LogEntry*L){
2     bool v=IsLogsEmpty(L);
3
4     if (v)
5     {
6         return 0; // Return 0 if the list is empty
7     }
8     else
9     {
10        int counter=0;
11        while (L!=NULL)
12        {
13            counter++; // Increment counter for each node
14            L=L->next; // Move to the next node
15        }
16        return counter; // Return the total count

```

```

17     }
18 }

```

Functionality:

Takes the head pointer of the list as input. Returns 0 if the list is empty. Otherwise, it traverses the list from the head, incrementing a counter for each node until the end of the list is reached. Returns the final count.

4.8 Deleting a Log Entry by ID

The DelLogEntryByID function searches for a log entry with a specific ID and removes it from the singly linked list.

```

1 void DelLogEntryByID(LogEntry**L, int ID){
2     LogEntry *p; // Pointer to traverse the list
3     LogEntry *q; // Pointer to the node to be deleted
4     p=*L;
5
6     // Case 1: Deleting the head node
7     if(p != NULL && p->id == ID){
8         *L = p->next; // Head moves to the next node
9         free(p); // Free the old head
10        printf("Log with ID %d deleted.\n", ID);
11        return;
12    }
13
14    // Case 2: Deleting a node other than the head
15    while(p != NULL && p->next != NULL){
16        if(p->next->id == ID){
17            q = p->next; // Node to be deleted
18            p->next = p->next->next; // Skip over the node to delete
19            free(q); // Free the node
20            printf("Log with ID %d deleted.\n", ID);
21            return ; // Exit after deletion
22        }
23        p=p->next; // Move to the next node
24    }
25
26    // Case 3: ID not found
27    printf("\nID %d NOT found\n", ID);
28 }

```

Functionality:

Takes the head pointer of the list and the target ID as input. Checks if the head node has the target ID and handles deletion if it does. Otherwise, it traverses the list, keeping track of the current node (p). If the next node of the current node has the target ID, it updates the next pointer of the current node to bypass the node to be deleted and then frees the memory of the deleted node. If the end of the list is reached without finding the ID, it prints a "Not found" message.

4.9 Deleting the First Log Entry

The DelLogAtTheBeginning function removes the first log entry from the singly linked list.

```

1 void DelLogAtTheBeginning(LogEntry**L){
2     bool v=IsLogsEmpty(*L);
3
4     if (v)
5     {
6         printf("They Are empty\n"); // Cannot delete from an empty list
7         return;
8     }
9     else
10    {
11        LogEntry*temp=(*L); // Store the current head
12        (*L)=(*L)->next; // Head moves to the next node
13        free(temp); // Free the memory of the old head
14        printf("First log deleted.\n");
15    }
16 }

```

```

15         return;
16     }
17 }

```

Functionality:

Takes the head pointer of the list as input. Checks if the list is empty. If the list is not empty, it updates the head pointer to the second node and frees the memory of the original head node.

4.10 Deleting the Last Log Entry

The DelLogAtTheEnd function removes the last log entry from the singly linked list.

```

1 void DelLogAtTheEnd(LogEntry**L){
2     bool v=IsLogsEmpty(*L);
3
4     if (v)
5     {
6         printf("They Are empty\n"); // Cannot delete from an empty list
7         return;
8     }
9     // Special case: only one node in the list
10    else if ((*L)->next==NULL)
11    {
12        free((*L)); // Free the single node
13        (*L)=NULL; // List becomes empty
14        printf("Last log deleted.\n");
15        return;
16    }
17    else
18    {
19        LogEntry*Tem=(*L);
20        // Traverse to the second to last node
21        while (Tem->next->next!=NULL)
22        {
23            Tem=Tem->next;
24        }
25        LogEntry*fre=Tem->next; // The last node to be freed
26        Tem->next=NULL; // The second to last node becomes the new last node
27        free(fre); // Free the original last node
28        printf("Last log deleted.\n");
29        return;
30    }
31 }

```

Functionality:

Takes the head pointer of the list as input. Checks if the list is empty. Handles the special case where there is only one node in the list. Otherwise, it traverses the list to find the second-to-last node. It then sets the next pointer of the second-to-last node to NULL and frees the memory of the original last node.

4.11 Deleting a Log Entry by Timestamp

The DelLogByTimeStamp function searches for a log entry with a timestamp matching user input and removes it.

```

1 bool isValidDateTime(int year, int month, int day,int hour, int minutes, int seconds) {
2     // Validate year (assuming reasonable range)
3     if (year < 1900 || year > 2100) return false;
4
5     // Validate month
6     if (month < 1 || month > 12) return false;
7
8     // Validate day
9     if (day < 1 || day > 31) return false;
10
11    // Check months with 30 days
12    if ((month == 4 || month == 6 || month == 9 || month == 11) && day > 30)
13        return false;

```

```

14     if (hour >= 24 || hour < 0) // Corrected hour validation
15     {
16         return false;
17     }
18
19     // Check February (ignoring leap years for simplicity)
20     // A more robust check would account for leap years.
21     if (month == 2) {
22         // Basic check, doesn't handle leap years
23         if (day > 28) return false;
24     }
25
26     // Validate minutes and seconds
27     if (minutes < 0 || minutes > 59) return false;
28     if (seconds < 0 || seconds > 59) return false;
29
30     return true;
31 }
32
33 void DelLogByTimeStamp(LogEntry** L) {
34     if (*L == NULL) {
35         printf("Log list is empty!\n");
36         return;
37     }
38     int year, month, day, hour, minutes, seconds;
39     char tryAgain;
40     char temp [50];
41     printf("Enter the TimeStamp in this format:YYYY MM DD HH MM SS\n"); // Adjusted
42     prompt
43     do {
44         printf("Enter date and time (YYYY MM DD HH MM SS): ");
45         if (scanf("%d %d %d %d %d %d", &year, &month, &day, &hour, &minutes, &seconds
46         ) != 6) {
47             printf("Invalid input format. Please enter numbers.\n");
48             while (getchar() != '\n'); // Clear input buffer
49             continue;
50         }
51         while (getchar() != '\n'); // Clear input buffer
52         if (!isValidDateTime(year, month, day, hour, minutes, seconds)) {
53             printf("Invalid date or time values. Please try again.\n");
54             continue;
55         }
56
57         // Format the entered components into a string.
58         // IMPORTANT: This format needs to MATCH the format used by
59         getCurrentTimeString (%Y-%m-%d %H:%M:%S).
60         sprintf(temp, "%04d-%02d-%02d %02d:%02d:%02d", year, month, day, hour,
61         minutes, seconds);
62
63         printf("Searching for logs with timestamp containing: %s\n", temp);
64
65         printf("Do you want to enter another date? (y/n): ");
66         scanf(" %c", &tryAgain);
67
68         while (getchar() != '\n');
69     } while (tryAgain == 'y' || tryAgain == 'Y');
70
71     LogEntry *current = *L;
72     LogEntry *previous = NULL;
73
74     while (current != NULL) {
75         if (strstr(current->timestamp, temp) != NULL) { // Using strstr for
76         substring match
77             if (previous != NULL) {
78                 previous->next = current->next;
79             } else {
80                 *L = current->next; // Deleting the head
81             }
82
83             free(current);

```

```

82         printf("Log with timestamp containing '%s' deleted.\n", temp);
83         return; // Assuming only the first match is deleted
84     }
85     previous = current;
86     current = current->next;
87 }
88
89 printf("No log found with timestamp containing '%s'\n", temp);
90 }

```

Functionality:

Prompts the user for date and time components and validates the input. Formats the input into a string for comparison. Traverses the list and uses 'strstr' to find a log entry whose timestamp contains the formatted input string. If a match is found, it updates the 'next' pointer of the previous node to remove the current node. Handles the case of deleting the head node. Frees the memory of the deleted node. Prints a "Not found" message if no match is found.

4.12 Sorting Logs by Severity (Distribution Sort)

The `SortBySeverity` function sorts the singly linked list based on the severity level of the log entries ("INFO", "WARNING", "ERROR"). It uses a counting or distribution sort approach.

```

1 void SortBySeverity(LogEntry** head) {
2     if (*head == NULL) return;
3
4     // Pointers for the heads and tails of the three severity lists
5     LogEntry *info_head = NULL, *info_tail = NULL;
6     LogEntry *warn_head = NULL, *warn_tail = NULL;
7     LogEntry *error_head = NULL, *error_tail = NULL;
8
9     LogEntry* current = *head;
10    while (current != NULL) {
11        LogEntry* next = current->next; // Store the next node before modifying
12        current->next = NULL; // Detach the current node from the original list
13
14        if (strcmp(current->severity, "INFO") == 0) {
15            if (info_head == NULL) {
16                info_head = current;
17                info_tail = current;
18            } else {
19                info_tail->next = current;
20                info_tail = current;
21            }
22        }
23        else if (strcmp(current->severity, "WARNING") == 0) {
24            if (warn_head == NULL) {
25                warn_head = current;
26                warn_tail = current;
27            } else {
28                warn_tail->next = current;
29                warn_tail = current;
30            }
31        }
32        else { // Assuming anything else is ERROR
33            if (error_head == NULL) {
34                error_head = current;
35                error_tail = current;
36            } else {
37                error_tail->next = current;
38                error_tail = current;
39            }
40        }
41        current = next; // Move to the next node in the original list
42    }
43
44    // Reconnect the three lists in the desired order: INFO -> WARNING -> ERROR
45    *head = NULL; // Start with an empty list
46    LogEntry** tail = head;

```

```

47
48     if (info_head) {
49         *tail = info_head;
50         tail = &info_tail->next;
51     }
52
53     if (warn_head) {
54         *tail = warn_head;
55         tail = &warn_tail->next;
56     }
57
58     if (error_head) {
59         *tail = error_head;
60     }
61 }

```

Functionality:

Traverses the original linked list. For each node, it checks its severity level ("INFO", "WARNING", or "ERROR"). It moves each node to one of three separate lists: an INFO list, a WARNING list, and an ERROR list. After processing all nodes, it concatenates the three lists in the order INFO, then WARNING, then ERROR, by updating the `next` pointers of the tails of the preceding lists.

This is a stable sort, meaning the relative order of logs with the same severity is preserved.

4.13 Reversing the Linked List

The `ReverseLogs` function reverses the order of the nodes in the singly linked list.

```

1 void ReverseLogs(LogEntry**H){
2     // If the list is empty or has only one node, no reversal is needed
3     if (IsLogsEmpty(*H) || (*H)->next == NULL)
4     {
5         return;
6     }
7
8     LogEntry* L1 = *H; // Pointer to the current node (initially head)
9     LogEntry* L2 = L1->next; // Pointer to the next node
10    L1->next = NULL; // The original head will become the new tail, so its next is
    NULL
11    LogEntry* L3 = NULL; // Pointer to the node after L2
12
13    while (L2 != NULL)
14    {
15        L3 = L2->next; // Store the next node before changing L2->next
16        L2->next = L1; // Reverse the pointer: L2's next points to L1
17        L1 = L2; // Move L1 forward to the current node (which was L2)
18        L2 = L3; // Move L2 forward to the next node (which was stored in L3)
19    }
20    *H = L1; // L1 is now pointing to the new head (the original last node)
21 }

```

Functionality:

Uses three pointers (L1, L2, L3) to keep track of the current, next, and next-next nodes during traversal. It iteratively changes the `next` pointer of each node to point to its *previous* node. The original head's `next` pointer is set to `NULL`. The head pointer of the list is updated to point to the original last node, which becomes the new head.

4.14 Searching for a Log Entry by ID

The `serchLogById` function searches for a log entry with a specific ID and prints its details if found.

```

1 LogEntry* serchLogById(LogEntry*L, int ID){
2     if (L == NULL)
3     {
4         printf("it is empty\n"); // List is empty, ID cannot be found
5         return NULL;
6     }
7     while (L != NULL)

```

```

8      {
9          if (L->id == ID)
10         {
11             printf("it was found\n");
12             displayLogEntryColumn(L); // Display the found log entry
13             return L; // Return pointer to the found node
14         }
15         L = L->next; // Move to the next node
16     }
17     printf("Not found \n"); // ID not found after traversing the list
18     getchar(); // Consume leftover newline (might not be needed depending on prior
19     input)
20     return NULL; // Return NULL if not found
}

```

Functionality:

Takes the head pointer of the list and the target ID as input. Traverses the list from the head. Compares the id of each node with the target ID. If a match is found, it prints a "found" message, displays the log entry details, and returns a pointer to the found node. If the end of the list is reached without finding the ID, it prints a "Not found" message and returns NULL.

4.15 Generating Multiple Log Entries

The GenLog function allows the user to generate a specified number of log entries and add them to the singly linked list. It uses InsertLogEntryByPos to add logs, which in turn calls InsertLogEntry.

```

1 void GenLog(LogEntry** L, int size) {
2     int i;
3     for (i = 0; i < size; i++) {
4         // Insert each new log. The position input will be handled inside
5         InsertLogEntryByPos.
6         printf("\nEntering details for log %d:\n", i + 1);
7         InsertLogEntryByPos(L);
8     }
9     // Display all logs after generation is complete
10    printf("\n--- All Generated Logs ---\n");
11    DisplayLogs(*L);
12 }

```

Functionality:

Takes the head pointer of the list and the number of logs to generate as input. Uses a loop to repeatedly call InsertLogEntryByPos to create and insert the specified number of log entries. After generating all logs, it calls DisplayLogs to show the contents of the list.

Bidirectional Linked Lists

5 Introduction

Bidirectional linked lists, also known as doubly linked lists, are a more complex form of linked list compared to singly linked lists. Each node in a bidirectional linked list contains three parts:

The data of the node (the log entry details). A pointer ('next') that points to the next node in the sequence. A pointer ('prev') that points to the previous node in the sequence.

This additional 'prev' pointer allows for traversal in both forward and backward directions, which can simplify the implementation of certain operations, such as deletion and reversal, compared to singly linked lists.

6 Log Entry Structure (Bidirectional Linked List)

The structure used for log entries in the bidirectional linked list is BLogEntry.

```

1 struct BLogEntry {
2     int id; // Auto-incremented identifier
3     char message[100]; // Log content
4     struct BLogEntry *next; // Pointer to next entry
5     char severity[10]; // INFO/WARNING/ERROR
6     char timestamp[50]; // Formatted timestamp
7     struct BLogEntry *prev; // Pointer to previous entry
8 };
9 typedef struct BLogEntry BLogEntry;

```

Structure Breakdown:

‘id’: An integer to uniquely identify each log entry. ‘message’: A character array to store the log message content. A fixed size of 100 is used. ‘next’: A pointer that points to the subsequent log entry in the list. ‘severity’: A character array to store the severity level of the log (e.g., “INFO”, “WARNING”, “ERROR”). A fixed size of 10 is used. ‘timestamp’: A character array to store the timestamp of when the log was created, formatted as a string. A fixed size of 50 is used. ‘prev’: A pointer that points to the previous log entry in the list.

The ‘typedef’ statement creates an alias ‘BLogEntry’ for ‘struct BLogEntry’.

7 Bidirectional Linked List Implementation Details

7.1 Creating a New Bidirectional Log Entry Node

The CreateBLogEntryBI function allocates memory for a new BLogEntry node and initializes its fields, including the ‘prev’ pointer.

```

1 BLogEntry*CreateBLogEntryBI(){
2     BLogEntry* temp = (BLogEntry*)malloc(sizeof(BLogEntry));
3     if (temp == NULL) {
4         printf("\nMemory Allocation Failed\n");
5         return NULL;
6     }
7
8     temp->id = ID++;
9     temp->next = NULL;
10    temp->prev = NULL; // Initialize prev pointer
11    char msg[100];
12    char msg2[100];
13    char* temp3 = getCurrentTimeString();
14
15    if (temp3) {
16        strcpy(temp->timestamp, temp3);
17        free(temp3);
18    } else {
19        strcpy(temp->timestamp, "ERROR");
20    }
21
22    printf("\nWhat is the message?\n");
23    fgets(msg2, sizeof(msg2), stdin);
24    size_t len = strlen(msg2);
25    if (len > 0 && msg2[len - 1] == '\n') {
26        msg2[len - 1] = '\0';
27    }
28    strcpy(temp->message, msg2);
29
30    do {
31        printf("What is the level of severity (INFO, WARNING, ERROR)?\n");
32        if (fgets(msg, sizeof(msg), stdin) != NULL) {
33            size_t len = strlen(msg);
34            if (len > 0 && msg[len - 1] == '\n') {
35                msg[len - 1] = '\0';
36            }
37            strcpy(temp->severity, msg);
38        } else {
39            printf("Error reading severity level.\n");
40            free(temp);
41            return NULL;
42        }

```



```

43 } while (strcmp(temp->severity, "INFO") != 0 && strcmp(temp->severity, "WARNING") !=
44         0 && strcmp(temp->severity, "ERROR") != 0);
45 return temp;
}

```

Functionality:

Allocates memory for a new `BLogEntry` using `malloc`. Increments a global ID counter and assigns it to the new entry's id. Sets both the `next` and `prev` pointers to `NULL`. Calls `getCurrentTimeString` to get the current timestamp string and copies it to the `timestamp` field. Prompts the user for the log message and severity level, reading input using `fgets` and handling potential newline characters. Validates the entered severity level. Returns a pointer to the newly created `BLogEntry` node, or `NULL` if memory allocation or input failed.

7.2 Checking if the Bidirectional List is Empty

The `IsLogsEmpty` function (reused for both list types) checks if the bidirectional linked list is empty.

```

1 bool IsLogsEmpty(BLogEntry*L){
2     return (L==NULL);
3 }

```

Functionality:

Takes the head pointer of the list (`L`) as input. Returns `true` if the head pointer is `NULL`, and `false` otherwise.

7.3 Adding a Log Entry at the Beginning (Bidirectional)

The `AddBLogEntryAtTheBeginning` function inserts a new log entry at the front of the bidirectional linked list.

```

1 void AddBLogEntryAtTheBeginning(BLogEntry**L){
2
3     bool v=IsLogsEmpty(*L);
4
5     BLogEntry*temp=CreateBLogEntryBI();
6
7     if (temp==NULL)
8     {
9         printf("Allocation of Mem failed\n");
10        return;
11    }
12
13    if (v)
14    {
15        (*L)=temp;
16        return;
17    }
18    else
19    {
20        temp->next=(*L);
21        (*L)->prev = temp; // Update prev pointer of the original head
22        (*L)=temp;
23        return;
24    }
25 }

```

Functionality:

Calls `CreateBLogEntryBI` to get a new log entry node. If the list is empty, the new node becomes the head. If the list is not empty, the new node's `next` pointer is set to the current head, the original head's `prev` pointer is set to the new node, and the list's head pointer is updated to the new node.

7.4 Adding a Log Entry at the End (Bidirectional)

The `AddBLogEntryAtTheEndBI` function appends a new log entry to the end of the bidirectional linked list.

```

1 void AddBlogEntryAtTheEndBI (BLogEntry**L){
2     bool v=IsLogsEmpty(*L);
3
4     BlogEntry*temp=CreateBlogEntryBI();
5
6     if (temp==NULL)
7     {
8         printf("Addition Failed\n");
9         return;
10    }
11
12    if (v)
13    {
14        (*L)=temp;
15    }
16
17    else
18    {
19        BlogEntry*Tem=(*L);
20        while (Tem->next!=NULL)
21        {
22            Tem=Tem->next;
23        }
24        Tem->next=temp;
25        temp->prev=Tem; // Update prev pointer of the new node
26        return;
27    }
28 }

```

Functionality:

Calls CreateBlogEntryBI to get a new log entry node. If the list is empty, the new node becomes the head. If the list is not empty, it traverses the list to find the last node, sets the last node's next pointer to the new node, and sets the new node's prev pointer to the original last node.

7.5 Inserting a Log Entry at a Specific Position (Bidirectional)

The InsertBlogEntryBI function inserts a new log entry at a specified position within the bidirectional linked list. InsertBlogEntryByPosBI is a helper to get the position from the user.

```

1 void InsertBlogEntryBI(BLogEntry** L, int pos) {
2     int counter = 1;
3     BlogEntry* temp = (BlogEntry*)malloc(sizeof(BLogEntry));
4     if (temp == NULL) {
5         printf("\nMemory Allocation Failed\n");
6         return;
7     }
8
9     temp->id = ID++;
10    temp->next = NULL;
11    temp->prev = NULL; // Initialize prev pointer
12    char msg[100];
13    char msg2[100];
14    char* temp3 = getCurrentTimeString();
15
16    if (temp3) {
17        strcpy(temp->timestamp, temp3);
18        free(temp3);
19    } else {
20        strcpy(temp->timestamp, "ERROR");
21    }
22
23    printf("\nWhat is the message?\n");
24    fgets(msg2, sizeof(msg2), stdin);
25    size_t len = strlen(msg2);
26    if (len > 0 && msg2[len - 1] == '\n') {
27        msg2[len - 1] = '\0';
28    }
29    strcpy(temp->message, msg2);
30 }

```

```

31 do {
32     printf("What is the level of severity (INFO, WARNING, ERROR)?\n");
33     if (fgets(msg, sizeof(msg), stdin) != NULL) {
34         size_t len = strlen(msg);
35         if (len > 0 && msg[len - 1] == '\n') {
36             msg[len - 1] = '\0';
37         }
38         strcpy(temp->severity, msg);
39     } else {
40         printf("Error reading severity level.\n");
41         free(temp);
42         return;
43     }
44 } while (strcmp(temp->severity, "INFO") != 0 && strcmp(temp->severity, "WARNING") !=
0 && strcmp(temp->severity, "ERROR") != 0);
45
46 if (pos == 1 || *L == NULL) {
47     temp->next = *L;
48     if (*L != NULL) {
49         (*L)->prev = temp; // Update prev of original head
50     }
51     temp->prev=NULL; // New head has no previous
52     *L = temp;
53     return;
54 }
55
56 BLogEntry* current = *L;
57 BLogEntry* previous = NULL;
58
59 while (current != NULL && counter < pos) {
60     previous = current;
61     current = current->next;
62     counter++;
63 }
64
65 if (counter == pos) {
66     temp->next = current;
67     temp->prev = previous;
68     if (previous != NULL) {
69         previous->next = temp;
70     }
71     if (current != NULL) {
72         current->prev = temp; // Update prev of the node after insertion point
73     }
74 } else {
75     printf("The position is out of range.\n");
76     free(temp);
77 }
78 }
79
80 void InsertBLogEntryByPosBI(BLogEntry** L) {
81     int pos;
82     printf("Enter the position: ");
83     scanf("%d", &pos);
84     getchar(); // Consume leftover '\n'
85     InsertBLogEntryBI(L, pos);
86 }

```

Functionality:

Calls `CreateBLogEntryBI` to get a new log entry node. Handles insertion at the beginning as a special case, updating the `prev` pointer of the original head. For other positions, it traverses the list to find the node *before* the insertion point. Updates the `next` pointer of the previous node and the `prev` pointer of the current node (the one after the insertion point) to link the new node into the list. If the specified position is out of range, it prints an error and frees the allocated memory.

7.6 Displaying All Log Entries (Bidirectional)

The `DisplayLogs` function (overloaded or a separate function in practice) iterates through the bidirectional linked list and prints the details of each log entry. The provided code includes a version that takes

a ‘pos’ argument, likely for highlighting a specific entry during navigation.

```
1 void displayBLogEntryColumn(BLogEntry *entry) {
2     if (entry==NULL)
3     {
4         printf("it is Empty \n");
5         return;
6     }
7     printf("%-5d | %-32s | %-10s | %-100s\n",
8           entry->id, entry->timestamp, entry->severity, entry->message);
9 }
10 void displaySelBLogEntryColumn(BLogEntry *entry) {
11     printf("%-5d | %-32s | %-10s | %-100s\n",
12           entry->id, entry->timestamp, entry->severity, entry->message);
13 }
14
15 void DisplayLogs(BLogEntry* L,int pos) { % Note: This function name conflicts with the
16     singly list version
17     if (L == NULL) {
18         printf("Logs are empty\n");
19         return;
20     }
21     displayLogHeader();
22     int counter=1;
23     while (L != NULL) {
24         if (pos==counter)
25         {
26             displaySelBLogEntryColumn(L); % Highlight selected entry
27         }
28         else
29         {
30             displayBLogEntryColumn(L); % Display normally
31         }
32         L = L->next;
33         counter++;
34     }
35 }
```

Functionality:

Takes the head pointer of the list and an optional position to highlight as input. Checks if the list is empty. Prints a header. Traverses the list, printing the details of each entry. If the current entry’s position matches the ‘pos’ argument, it uses ‘displaySelBLogEntryColumn’ for highlighting.

7.7 Getting the Number of Log Entries (Bidirectional)

The LogSize function (reused) calculates the size of the bidirectional linked list.

```
1 int LogSize(BLogEntry*L){
2
3     bool v=IsLogsEmpty(L);
4
5     if (v)
6     {
7         return 0;
8     }
9     else
10    {
11        int counter=0;
12        while (L!=NULL)
13        {
14            counter++;
15            L=L->next;
16        }
17        return counter;
18    }
19 }
```

Functionality:

Takes the head pointer of the list as input. Returns 0 if the list is empty. Otherwise, it traverses the list using the ‘next’ pointers and counts the nodes.

7.8 Deleting a Log Entry by ID (Bidirectional)

The DelBLogEntryByID function searches for a log entry by ID and removes it from the bidirectional list.

```
1 void DelBLogEntryByID(BLogEntry**L, int ID){
2     BLogEntry *p;
3     p=*L;
4
5     % Case 1: Deleting the head node
6     if(p != NULL && p->id == ID){
7         *L = p->next;
8         if (*L != NULL) {
9             (*L)->prev = NULL; % Update prev of the new head
10        }
11        free(p);
12        printf("Log with ID %d deleted.\n", ID);
13        return;
14    }
15
16    % Case 2: Deleting a node other than the head
17    while(p != NULL && p->next != NULL){
18        if(p->next->id == ID){
19            BLogEntry* node_to_delete = p->next;
20            p->next = node_to_delete->next;
21            if (node_to_delete->next != NULL) {
22                node_to_delete->next->prev = p; % Update prev of the node after deletion
23            }
24            free(node_to_delete);
25            printf("Log with ID %d deleted.\n", ID);
26            return ;
27        }
28        p=p->next;
29    }
30
31    % Case 3: ID not found
32    printf("\nID %d NOT found\n", ID);
33 }
```

Functionality:

Takes the head pointer and target ID as input. Handles deletion of the head node, updating the 'prev' pointer of the new head. For other nodes, it traverses the list. When the node to delete is found, it updates the 'next' pointer of the previous node and the 'prev' pointer of the next node to bypass and remove the target node. Frees the memory of the deleted node. Prints a "Not found" message if the ID is not found.

7.9 Deleting the First Log Entry (Bidirectional)

The DelLogAtTheBeginning function (reused) removes the first log entry from the bidirectional list.

```
1 void DelLogAtTheBeginning(BLogEntry**L){
2
3     bool v=IsLogsEmpty(*L);
4
5     if (v)
6     {
7         printf("They Are empty\n");
8         return;
9     }
10
11    else
12    {
13        BLogEntry*temp=(*L);
14        (*L)=(*L)->next;
15        if (*L != NULL) {
16            (*L)->prev = NULL; % Update prev of the new head
17        }
18        free(temp);
19        return;
20    }
21 }
```

Functionality:

Takes the head pointer as input. Checks for an empty list. If not empty, updates the head pointer to the second node and sets the new head's 'prev' pointer to 'NULL'. Frees the memory of the original head node.

7.10 Deleting the Last Log Entry (Bidirectional)

The DelLogAtTheEnd function (reused) removes the last log entry from the bidirectional list.

```
1 void DelLogAtTheEnd(BLogEntry**L){
2
3     bool v=IsLogsEmpty(*L);
4
5     if (v)
6     {
7         printf("They Are empty\n");
8         return;
9     }
10
11     else if ((*L)->next==NULL) % Special case: only one node
12     {
13         free((*L));
14         (*L)=NULL;
15         return;
16     }
17
18     else
19     {
20         BLogEntry*Tem=(*L);
21         % Traverse to the second to last node
22         while (Tem->next->next!=NULL)
23         {
24             Tem=Tem->next;
25         }
26         BLogEntry*fre=Tem->next; % The last node to be freed
27         Tem->next=NULL; % The second to last node becomes the new last node
28         % No need to update fre->prev as it's being freed
29         free(fre);
30         return;
31     }
32 }
```

Functionality:

Takes the head pointer as input. Checks for an empty list or a single-node list. For lists with more than one node, it traverses to the second-to-last node, sets its 'next' pointer to 'NULL', and frees the memory of the last node.

7.11 Deleting a Log Entry by Position (Bidirectional)

The DelLogByPos function deletes a log entry at a specified position in the bidirectional list.

```
1 void DelLogByPos(BLogEntry** L, int pos) {
2     % Check for empty list
3     if (*L == NULL) {
4         printf("List is empty!\n");
5         return;
6     }
7
8     int list_size = LogSize(*L);
9
10    % Validate position
11    if (pos < 1 || pos > list_size) {
12        printf("Position %d is out of range (1-%d)\n", pos, list_size);
13        return;
14    }
15
16    % Handle special cases
17    if (pos == 1) {
18        DelLogAtTheBeginning(L);
19        return;
20    }
21 }
```

```

20 }
21 if (pos == list_size) {
22     DelLogAtTheEnd(L);
23     return;
24 }
25
26 % General case: delete node in the middle
27 BLogEntry* current = *L;
28 int counter = 1;
29
30 % Traverse to the node at the specified position
31 while (current != NULL && counter < pos) {
32     current = current->next;
33     counter++;
34 }
35
36 if (current == NULL) {
37     printf("Error: Invalid position reached\n");
38     return;
39 }
40
41 % Update next pointer of previous node
42 if (current->prev != NULL) {
43     current->prev->next = current->next;
44 }
45
46 % Update prev pointer of next node (if exists)
47 if (current->next != NULL) {
48     current->next->prev = current->prev;
49 }
50
51 % Free the node
52 free(current);
53 printf("Log at position %d deleted.\n", pos);
54 }

```

Functionality:

Takes the head pointer and the position to delete as input. Handles empty list and out-of-range position errors. Calls 'DelLogAtTheBeginning' or 'DelLogAtTheEnd' for the respective cases. For deletion in the middle, it traverses to the node at the specified position. Updates the 'next' pointer of the previous node and the 'prev' pointer of the next node to remove the current node from the list. Frees the memory of the deleted node.

7.12 Deleting a Log Entry by Timestamp (Bidirectional)

The DelLogByTimeStamp function (reused) searches for a log entry by timestamp and removes it from the bidirectional list.

```

1 bool isValidDateTime(int year, int month, int day, int hour, int minutes, int seconds) {
2     % Validate year (assuming reasonable range)
3     if (year < 1900 || year > 2100) return false;
4
5     % Validate month
6     if (month < 1 || month > 12) return false;
7
8     % Validate day
9     if (day < 1 || day > 31) return false;
10
11     % Check months with 30 days
12     if ((month == 4 || month == 6 || month == 9 || month == 11) && day > 30)
13         return false;
14     if (hour >= 24 || hour < 0)
15     {
16         return false;
17     }
18
19     % Check February (ignoring leap years for simplicity)
20     if (month == 2 && day > 28) return false;
21
22     % Validate minutes and seconds
23     if (minutes < 0 || minutes > 59) return false;

```

```

24     if (seconds < 0 || seconds > 59) return false;
25
26     return true;
27 }
28
29 void DelLogByTimeStamp(BLogEntry** L) { % Note: This function name conflicts with the
    singly list version
30     if (*L == NULL) {
31         printf("Log list is empty!\n");
32         return;
33     }
34     int year, month, day, hour, minutes, seconds;
35     char tryAgain;
36     char temp [50];
37     printf("Enter the TimeStamp in this format:YYYY MM DD HH MM SS\n"); % Adjusted
    prompt
38     do {
39         printf("Enter date and time (YYYY MM DD HH MM SS): ");
40         if (scanf("%d %d %d %d %d %d", &year, &month, &day, &hour, &minutes, &seconds) !=
        6) {
41             printf("Invalid input format. Please enter numbers.\n");
42             while (getchar() != '\n'); % Clear input buffer
43             continue;
44         }
45
46         while (getchar() != '\n'); % Clear input buffer
47
48         if (!isValidDateTime(year, month, day, hour, minutes, seconds)) {
49             printf("Invalid date or time values. Please try again.\n");
50             continue;
51         }
52
53         % Format the entered components into a string.
54         % IMPORTANT: This format needs to MATCH the format used by getCurrentTimeString
        (%Y-%m-%d %H:%M:%S).
55         sprintf(temp, "%04d-%02d-%02d %02d:%02d:%02d", year, month, day, hour, minutes,
        seconds);
56
57         printf("Searching for logs with timestamp containing: %s\n", temp);
58
59         printf("Do you want to enter another date? (y/n): ");
60         scanf(" %c", &tryAgain);
61
62         while (getchar() != '\n');
63
64     } while (tryAgain == 'y' || tryAgain == 'Y');
65
66     BLogEntry *current = *L;
67
68     while (current != NULL) {
69         if (strstr(current->timestamp, temp) != NULL) { % Using strstr for substring
        match
70             if (current->prev != NULL) {
71                 current->prev->next = current->next;
72             } else {
73                 *L = current->next; % Deleting the head
74             }
75
76             if (current->next != NULL) {
77                 current->next->prev = current->prev;
78             }
79
80             free(current);
81             printf("Log with timestamp containing '%s' deleted.\n", temp);
82             return; % Assuming only the first match is deleted
83         }
84         current = current->next;
85     }
86
87     printf("No log found with timestamp containing '%s'\n", temp);
88 }

```

Functionality:

Prompts the user for date and time components and validates the input. Formats the input into a string for comparison. Traverses the list and uses 'strstr' to find a log entry whose timestamp contains the formatted input string. If a match is found, it updates the 'next' pointer of the previous node and the 'prev' pointer of the next node to remove the current node. Handles the case of deleting the head node. Frees the memory of the deleted node. Prints a "Not found" message if no match is found.

8 Memory Management

This section describes how memory is allocated, managed, and deallocated for log entries to prevent memory leaks.

8.1 Log Creation and Allocation

The `CreateLogEntry` (for singly lists) and `CreateBLogEntryBI` (for bidirectional lists) functions are responsible for allocating memory for new log entry nodes using `malloc` and initializing their fields.

```
1 % See code for CreateLogEntry and CreateBLogEntryBI in Chapter 1 and Chapter 2
   respectively.
```

Functionality:

Allocates memory using `malloc`. Initializes node fields, including 'next' and 'prev' (for bidirectional). Returns a pointer to the new node or `NULL` on allocation failure.

8.2 Log Deletion and Deallocation

Log deletion functions (`DelLogEntryByID`, `DelLogAtTheBeginning`, `DelLogAtTheEnd`, `DelLogByTimeStamp`, `DelLogByPos`) are responsible for freeing the memory of deleted nodes using the `free` function to prevent memory leaks. When a node is removed, its memory is returned to the system.

8.3 Handling Memory Errors

The system includes checks for memory allocation failures by verifying if the return value of `malloc` is `NULL`. If allocation fails, an error message is printed, and the operation is typically aborted to prevent the program from crashing due to accessing invalid memory.

9 Sorting Algorithms

This section covers the sorting algorithms used to organize log entries based on various criteria.

9.1 Severity-Based Sorting

Severity-based sorting is implemented for both singly and bidirectional linked lists using a distribution sort approach.

Singly Linked List Implementation (Section 4.12)

```
1 % See code for SortBySeverity in Chapter 1.
```

Bidirectional Linked List Implementation (Section ??)

```
1 % See code for SortBySeverityBI in Chapter 2.
```

9.2 Merge Sort Implementation (Bidirectional)

Merge Sort is implemented for sorting the bidirectional linked list by ID.

```
1 % See code for SortLogsByIdBI and merge2ListBI in Chapter 2.
```

Functionality:

Recursively splits the list into sublists. Merges sorted sublists while maintaining sorted order and updating 'next' and 'prev' pointers.

9.3 Other Sorting Criteria

While severity and ID based sorting are implemented, other criteria like timestamp or message content could also be used for sorting, potentially requiring different sorting algorithms or comparison functions.

10 Error Handling

This section outlines the error handling mechanisms implemented in the system.

10.1 Timestamp Generation Errors

The `getCurrentTimeString` function includes checks for errors during time retrieval and formatting, returning NULL if an error occurs. Callers of this function should handle the NULL return value, as seen in the log creation functions.

10.2 Memory Allocation Errors

As discussed in Section 10.2, the system checks for and handles memory allocation failures.

10.3 Invalid User Input

As discussed in Section 10.3, the system validates user input to prevent errors and unexpected behavior.

11 Search Operations

This section details how log entries can be searched within the system.

11.1 ID-Based Search

ID-based search is implemented for both singly and bidirectional linked lists.

Singly Linked List Implementation (Section 4.14)

```
1 % See code for serchLogById in Chapter 1.
```

Bidirectional Linked List Implementation (Section ??)

```
1 % See code for serchLogById in Chapter 2.
```

11.2 Timestamp-Based Search

Timestamp-based search is performed by traversing the list and comparing timestamps. The deletion function `DelLogByTimeStamp` (Section 12.3) demonstrates the timestamp comparison logic using 'strstr'. A dedicated search function would use similar logic but return matching nodes instead of deleting them.

11.3 Message Content Search

Message content search is implemented for the bidirectional linked list using the `SerchByKeyWord` function.

```
1 % See code for SerchByKeyWord in Chapter 2.
```

Functionality:

Prompts for a keyword. Traverses the list and uses ‘`strstr`’ to find the keyword in log messages.
Displays matching logs.

12 Deletion Methods

This section describes the different ways log entries can be removed from the system.

12.1 Deleting by ID

Deletion by ID is implemented for both singly and bidirectional linked lists.

Singly Linked List Implementation (Section 4.8)

```
1 % See code for DelLogEntryByID in Chapter 1.
```

Bidirectional Linked List Implementation (Section 7.8)

```
1 % See code for DelBLogEntryByID in Chapter 2.
2 \end{lstlisting}
3
4 \subsection{Deleting the First Entry}
5 \label{sub:delete_first_entry}
6 Deleting the first entry is implemented for both singly and bidirectional linked lists.
7
8 \subsubsection*{Singly Linked List Implementation (Section \ref{ssub:del_log_beginning})}
9 \begin{lstlisting}
10 % See code for DelLogAtTheBeginning in Chapter 1.
```

⌋

Bidirectional Linked List Implementation (Section 7.9)

```
1 % See code for DelLogAtTheBeginning in Chapter 2.
```

⌋

12.2 Deleting the Last Entry

Deleting the last entry is implemented for both singly and bidirectional linked lists.

Singly Linked List Implementation (Section 4.10)

```
1 % See code for DelLogAtTheEnd in Chapter 1.
```

⌋

Bidirectional Linked List Implementation (Section 7.10)

```
1 % See code for DelLogAtTheEnd in Chapter 2.
```

⌋

12.3 Timestamp-Based Deletion

Timestamp-based deletion is implemented for both singly and bidirectional linked lists.

Singly Linked List Implementation (Section 4.11)

```
1 % See code for DelLogByTimeStamp in Chapter 1.
```

⋮

Bidirectional Linked List Implementation (Section 7.12)

```
1 % See code for DelLogByTimeStamp in Chapter 2.
```

⋮

12.4 Deleting by Position (Bidirectional)

Deletion by position is implemented for the bidirectional linked list.

```
1 % See code for DelLogByPos in Chapter 2.
```

⋮

13 Utility Functions

This section covers general utility functions used throughout the system.

13.1 List Reversal

List reversal is implemented for both singly and bidirectional linked lists.

Singly Linked List Implementation (Section 4.13)

```
1 % See code for ReverseLogs in Chapter 1.
```

⋮

Bidirectional Linked List Implementation (Section ??)

```
1 % See code for ReverseLogsBI in Chapter 2.
```

⋮

13.2 Getting List Size

Getting the list size is implemented for both singly and bidirectional linked lists.

Singly Linked List Implementation (Section 4.7)

```
1 % See code for LogSize in Chapter 1.
```

⋮

Bidirectional Linked List Implementation (Section 7.7)

```
1 % See code for LogSize in Chapter 2.
```

⋮

13.3 Checking if List is Empty

Checking if the list is empty is implemented for both singly and bidirectional linked lists.

Singly Linked List Implementation (Section 4.2)

```
1 % See code for IsLogsEmpty in Chapter 1.
```

l

Bidirectional Linked List Implementation (Section 7.2)

```
1 % See code for IsLogsEmpty in Chapter 2.
```

l

13.4 Generating Logs

Generating logs is implemented for both singly and bidirectional linked lists.

Singly Linked List Implementation (Section 4.15)

```
1 % See code for GenLog in Chapter 1.
```

l

Bidirectional Linked List Implementation (Section ??)

```
1 % See code for GenLogBI in Chapter 2.
```

l

13.5 Navigating Logs (Bidirectional)

Navigation functions are implemented for the bidirectional linked list.

```
1 % See code for MoveForward, MoveBackward, and ReturnToBeginning in Chapter 2.
```

l

14 Algorithmic Complexity Analysis

This section provides an analysis of the time and space complexity for key algorithms and operations.

14.1 Understanding Big-O Notation

Big-O notation is a mathematical notation that describes the limiting behavior of a function when the argument tends towards a particular value or infinity. In the context of algorithms, it is used to classify them according to how their running time or space requirements grow as the input size grows. It provides an upper bound on the growth rate.

14.2 Singly Linked List Operation Complexities

Let n be the number of log entries (nodes) in the singly linked list. The following is a complexity analysis for the core operations discussed in Chapter 1.

Time Complexity

Time complexity measures how the execution time of an algorithm grows with the input size.

CreateLogEntry: $O(1)$ IsLogsEmpty: $O(1)$ AddLogEntryAtTheBeginning: $O(1)$ AddLogEntryAtTheEnd: $O(n)$
 InsertLogEntry (and InsertLogEntryByPos): $O(n)$ DisplayLogs: $O(n)$ LogSize: $O(n)$
 DelLogEntryByID: $O(n)$ DelLogAtTheBeginning: $O(1)$ DelLogAtTheEnd: $O(n)$
 DelLogByTimeStamp: $O(n \times L)$ SortBySeverity: $O(n)$ ReverseLogs: $O(n)$ serchLogById: $O(n)$
 GenLog: $O(size \times n)$

Space Complexity

Space complexity measures the amount of memory used by an algorithm.

For most operations: $O(1)$ SortBySeverity: $O(1)$ GenLog: $O(1)$

Summary of Singly Linked List Complexities:

Table 1: Singly Linked List Operation Complexities

Operation	Time Complexity (Worst Case)	Space Complexity
CreateLogEntry	$O(1)$	$O(1)$
IsLogsEmpty	$O(1)$	$O(1)$
AddLogEntryAtTheBeginning	$O(1)$	$O(1)$
AddLogEntryAtTheEnd	$O(n)$	$O(1)$
InsertLogEntry	$O(n)$	$O(1)$
DisplayLogs	$O(n)$	$O(1)$
LogSize	$O(n)$	$O(1)$
DelLogEntryByID	$O(n)$	$O(1)$
DelLogAtTheBeginning	$O(1)$	$O(1)$
DelLogAtTheEnd	$O(n)$	$O(1)$
DelLogByTimeStamp	$O(n \times L)$	$O(1)$
SortBySeverity	$O(n)$	$O(1)$
ReverseLogs	$O(n)$	$O(1)$
serchLogById	$O(n)$	$O(1)$
GenLog	$O(size \times n)$	$O(1)$

14.3 Bidirectional Linked List Operation Complexities

Let n be the number of log entries (nodes) in the bidirectional linked list. The following is a complexity analysis for the core operations discussed in Chapter 2.

Time Complexity

CreateBLogEntryBI: $O(1)$ IsLogsEmpty: $O(1)$ AddBLogEntryAtTheBeginning: $O(1)$
 AddBLogEntryAtTheEndBI: $O(n)$ InsertBLogEntryBI (and InsertBLogEntryByPosBI): $O(n)$
 DisplayLogs: $O(n)$ LogSize: $O(n)$ DelBLogEntryByID: $O(n)$ DelLogAtTheBeginning: $O(1)$
 DelLogAtTheEnd: $O(n)$ DelLogByPos: $O(n)$ DelLogByTimeStamp: $O(n \times L)$
 SortLogsByIdBI (Merge Sort): $O(n \log n)$ SortBySeverityBI: $O(n)$ ReverseLogsBI: $O(n)$
 serchLogById: $O(n)$ SerchByKeyword: $O(n \times L)$ GenLogBI: $O(size \times n)$ MoveForward, MoveBackward, ReturnToBeginning: $O(1)$

Space Complexity

For most operations: $O(1)$ SortLogsByIdBI (Merge Sort): $O(\log n)$ or $O(n)$ depending on implementation. SortBySeverityBI: $O(1)$ GenLogBI: $O(1)$

Summary of Bidirectional Linked List Complexities:

Table 2: Bidirectional Linked List Operation Complexities

Operation	Time Complexity (Worst Case)	Space Complexity
CreateBLogEntryBI	$O(1)$	$O(1)$
IsLogsEmpty	$O(1)$	$O(1)$
AddBLogEntryAtTheBeginning	$O(1)$	$O(1)$
AddBLogEntryAtTheEndBI	$O(n)$	$O(1)$
InsertBLogEntryBI	$O(n)$	$O(1)$
DisplayLogs	$O(n)$	$O(1)$
LogSize	$O(n)$	$O(1)$
DelBLogEntryById	$O(n)$	$O(1)$
DelLogAtTheBeginning	$O(1)$	$O(1)$
DelLogAtTheEnd	$O(n)$	$O(1)$
DelLogByPos	$O(n)$	$O(1)$
DelLogByTimeStamp	$O(n \times L)$	$O(1)$
SortLogsByIdBI (Merge Sort)	$O(n \log n)$	$O(\log n)$ or $O(n)$
SortBySeverityBI	$O(n)$	$O(1)$
ReverseLogsBI	$O(n)$	$O(1)$
serchLogById	$O(n)$	$O(1)$
SerchByKeyWord	$O(n \times L)$	$O(1)$
GenLogBI	$O(size \times n)$	$O(1)$
MoveForward, MoveBackward, ReturnToBeginning	$O(1)$	$O(1)$

14.4 Linked List Advantages

... (Discussion on advantages of linked lists will go here)

14.5 Merge Sort Complexity

Merge Sort is a comparison-based sorting algorithm that has a time complexity of $O(n \log n)$ in the worst, average, and best cases. Its efficiency stems from the divide-and-conquer approach. For linked lists, Merge Sort is particularly suitable because merging two sorted lists can be done efficiently by just changing pointer references, without requiring extra space for copying elements as in array-based merge sort.

15 Testing Methodology

15.1 Unit Test Cases

```

1 void TestInsertion() {
2     LogEntry* list = NULL;
3     AddLogEntryAtTheBeginning(&list);
4     assert(list != NULL);
5     assert(list->next == NULL);
6     FreeAllLogs(&list);
7 }
8
9 void TestSorting() {
10    LogEntry* list = CreateTestList(100);
11    SortLogsById(list, 100);
12    assert(IsSorted(list));
13    FreeAllLogs(&list);
14 }

```

Test Coverage:

Test Category	Coverage
Insertion	100%
• Deletion	100%
Sorting	100%
Edge Cases	100%

16 Performance Benchmarks

Operation Timings:

Operation	10,000 logs	100,000 logs
Insertion (head)	2ms	15ms
Merge Sort	45ms	650ms
Severity Sort	12ms	120ms
Search by ID	0.5ms	5ms

Circular Linked Lists

17 Introduction

Circular linked lists are a variation of linked lists where the last node points back to the first node, forming a circle. Unlike linear linked lists (singly or doubly), there is no 'NULL' pointer at the end of the list. This structure can be useful in scenarios where you need to cycle through elements continuously or where a starting point is arbitrary.

In a singly circular linked list, each node has a pointer to the next node, and the pointer of the last node points to the first node. In a doubly circular linked list, each node has pointers to both the next and previous nodes, and the 'next' pointer of the last node points to the first, and the 'prev' pointer of the first node points to the last. The implementation in 'circular.c' appears to use a singly circular structure based on the 'LogEntry' definition.

18 Log Entry Structure (Circular Linked List)

The structure used for log entries in the circular linked list implementation from 'circular.c' is also named `LogEntry`. While the structure definition itself is the same as the singly linked list, the way the 'next' pointers are managed creates the circular behavior.

```

1 struct LogEntry {
2     int id; // Auto-incremented identifier
3     char message[100]; // Log content
4     struct LogEntry *next; // Pointer to next entry
5     char severity[10]; // INFO/WARNING/ERROR
6     char timestamp[50]; // Formatted timestamp
7 };
8 typedef struct LogEntry LogEntry;

```

Structure Breakdown:

'id': An integer to uniquely identify each log entry. 'message': A character array to store the log message content. A fixed size of 100 is used. 'next': A pointer of type 'struct LogEntry *' that points to the subsequent log entry in the list. In a circular list, the 'next' pointer of the last node points back to the first node. 'severity': A character array to store the severity level of the log (e.g., "INFO", "WARNING", "ERROR"). A fixed size of 10 is used. 'timestamp': A character array to store the timestamp of when the log was created, formatted as a string. A fixed size of 50 is used.

The 'typedef' statement creates an alias 'LogEntry' for 'struct LogEntry'.

19 Circular Linked List Implementation Details

19.1 Creating a New Log Entry Node

The `CreateLogEntry` function (shared with the singly linked list implementation in ‘`circular.c`’) allocates memory for a new `LogEntry` node and initializes its fields.

```
1 LogEntry* CreateLogEntry(){
2     LogEntry* temp = (LogEntry*)malloc(sizeof(LogEntry));
3     if (temp == NULL) {
4         printf("\nMemory Allocation Failed\n");
5         return NULL;
6     }
7
8     temp->id = ID++;
9     temp->next = NULL; // Initially set to NULL, will be made circular upon insertion
10    char msg[100];
11    char msg2[100];
12    char* temp3 = getCurrentTimeString();
13
14    if (temp3) {
15        strcpy(temp->timestamp, temp3);
16        free(temp3);
17    } else {
18        strcpy(temp->timestamp, "ERROR");
19    }
20
21    printf("\nWhat is the message?\n");
22    fgets(msg2, sizeof(msg2), stdin);
23    size_t len = strlen(msg2);
24    if (len > 0 && msg2[len - 1] == '\n') {
25        msg2[len - 1] = '\0';
26    }
27    strcpy(temp->message, msg2);
28
29    do {
30        printf("What is the level of severity (INFO, WARNING, ERROR)?\n");
31        if (fgets(msg, sizeof(msg), stdin) != NULL) {
32            size_t len = strlen(msg);
33            if (len > 0 && msg[len - 1] == '\n') {
34                msg[len - 1] = '\0';
35            }
36            strcpy(temp->severity, msg);
37        } else {
38            printf("Error reading severity level.\n");
39            free(temp);
40            return NULL;
41        }
42    } while (strcmp(temp->severity, "INFO") != 0 && strcmp(temp->severity, "WARNING") !=
43              0 && strcmp(temp->severity, "ERROR") != 0);
44    return temp;
45 }
```

Functionality:

Allocates memory for a new <code>LogEntry</code> node.	Assigns a unique ID.	Initializes ‘next’ to ‘NULL’ (circularity is established during insertion).
for and reads the log message and severity level.	Returns the newly created node or ‘NULL’ on failure.	Prompts

19.2 Checking if the Circular List is Empty

The `IsLogsEmpty` function (shared) checks if the circular linked list is empty.

```
1 bool IsLogsEmpty(LogEntry* L) {
2     return (L == NULL);
3 }
```

Functionality:

Takes the head pointer as input.	Returns ‘true’ if the head pointer is ‘NULL’, indicating an empty list.
----------------------------------	---

19.3 Adding a Log Entry at the Beginning (Circular)

The `AddLogEntryAtTheBeginning` function (shared) inserts a new log entry at the front of the circular linked list.

```
1 void AddLogEntryAtTheBeginning(LogEntry** L) {
2     bool v = IsLogsEmpty(*L);
3     LogEntry* temp = CreateLogEntry();
4
5     if (temp == NULL) {
6         printf("Allocation of Mem failed\n");
7         return;
8     }
9
10    if (v) {
11        *L = temp;
12        temp->next = temp; // Make it circular
13    } else {
14        LogEntry* last = *L;
15        while (last->next != *L) {
16            last = last->next;
17        }
18        temp->next = *L;
19        *L = temp;
20        last->next = *L; // Update last node's next to new head
21    }
22 }
23 \end{lstlisting}
24
25 \textbf{Functionality}:
26 \begin{itemize}
27     \item Creates a new log entry node.
28     \item If the list is empty, the new node becomes the head and its 'next' pointer
29         points to itself.
30     \item If the list is not empty, it finds the last node (whose 'next' points to the
31         current head), sets the new node's 'next' to the current head, updates the head to
32         the new node, and updates the last node's 'next' to the new head.
33 \end{itemize}
34 \end{itemize}
35
36 \subsection{Adding a Log Entry at the End (Circular)}
37 \label{ssub:add_log_end_circular}
38 The \texttt{AddLogEntryAtTheEnd} function (shared) appends a new log entry to the end of
39 the circular linked list.
40
41 \begin{lstlisting}
42 void AddLogEntryAtTheEnd(LogEntry** L) {
43     bool v = IsLogsEmpty(*L);
44     LogEntry* temp = CreateLogEntry();
45
46     if (temp == NULL) {
47         printf("Addition Failed\n");
48         return;
49     }
50
51     if (v) {
52         *L = temp;
53         temp->next = temp; // Make it circular
54     } else {
55         LogEntry* last = *L;
56         while (last->next != *L) {
57             last = last->next;
58         }
59         last->next = temp;
60         temp->next = *L;
61     }
62 }
63 \end{lstlisting}
64
65 \textbf{Functionality}:
66 \begin{itemize}
67     \item Creates a new log entry node.
68     \item If the list is empty, the new node becomes the head and points to itself.
69     \item If the list is not empty, it finds the last node, sets the last node's 'next'
70         to the new node, and updates the new node's 'next' to the current head.
```

```

    ' to the new node, and sets the new node's 'next' to the head, maintaining
    circularity.
65 \end{itemize}
66
67 \subsection{Inserting a Log Entry at a Specific Position (Circular)}
68 \label{ssub:insert_log_position_circular}
69 The \texttt{InsertLogEntry} function (shared) inserts a new log entry at a specified
    position within the circular linked list. \texttt{InsertLogEntryByPos} is a helper
    to get the position from the user.
70
71 \begin{lstlisting}
72 void InsertLogEntry(LogEntry** L, int pos) {
73     int counter = 1;
74     LogEntry* temp = (LogEntry*)malloc(sizeof(LogEntry));
75     if (temp == NULL) {
76         printf("\nMemory Allocation Failed\n");
77         return;
78     }
79
80     temp->id = ID++;
81     temp->next = NULL;
82     char msg[100];
83     char msg2[100];
84     char* temp3 = getCurrentTimeString();
85
86     if (temp3) {
87         strcpy(temp->timestamp, temp3);
88         free(temp3);
89     } else {
90         strcpy(temp->timestamp, "ERROR");
91     }
92
93     printf("\nWhat is the message?\n");
94     fgets(msg2, sizeof(msg2), stdin);
95     size_t len = strlen(msg2);
96     if (len > 0 && msg2[len - 1] == '\n') {
97         msg2[len - 1] = '\0';
98     }
99     strcpy(temp->message, msg2);
100
101     do {
102         printf("What is the level of severity (INFO, WARNING, ERROR)?\n");
103         if (fgets(msg, sizeof(msg), stdin) != NULL) {
104             size_t len = strlen(msg);
105             if (len > 0 && msg[len - 1] == '\n') {
106                 msg[len - 1] = '\0';
107             }
108             strcpy(temp->severity, msg);
109         } else {
110             printf("Error reading severity level.\n");
111             free(temp);
112             return;
113         }
114     } while (strcmp(temp->severity, "INFO") != 0 && strcmp(temp->severity, "WARNING") !=
        0 && strcmp(temp->severity, "ERROR") != 0);
115
116     if (*L == NULL) {
117         *L = temp;
118         temp->next = *L; // Make it circular
119         return;
120     }
121
122     if (pos == 1) {
123         LogEntry* last = *L;
124         while (last->next != *L) {
125             last = last->next;
126         }
127         temp->next = *L;
128         *L = temp;
129         last->next = *L; // Update last node's next to new head
130         return;
131     }
132

```

```

133 LogEntry* current = *L;
134 LogEntry* previous = NULL;
135 int count = 1;
136
137 do {
138     if (count == pos) {
139         temp->next = current;
140         if (previous != NULL) {
141             previous->next = temp;
142         }
143         return;
144     }
145     previous = current;
146     current = current->next;
147     count++;
148 } while (current != *L && count <= pos);
149
150 if (count == pos) {
151     temp->next = *L;
152     previous->next = temp;
153 } else {
154     printf("The position is out of range.\n");
155     free(temp);
156 }
157 }
158
159 void InsertLogEntryByPos(LogEntry** L) {
160     int pos;
161     printf("Enter the position: ");
162     scanf("%d", &pos);
163     getchar();
164     InsertLogEntry(L, pos);
165 }

```

Functionality:

- Creates a new log entry node. Handles insertion into an empty list or at position 1 (beginning), updating pointers to maintain circularity. For insertion at other positions, it traverses the list to find the node before the desired position. Inserts the new node by updating the 'next' pointers of the previous node and the new node. Handles out-of-range positions.

;

19.4 Displaying All Log Entries (Circular)

The DisplayLogs function (shared) iterates through the circular linked list and prints the details of each log entry.

```

1 void DisplayLogs(LogEntry* L) {
2     if (L == NULL) {
3         printf("Logs are empty\n");
4         return;
5     }
6     displayLogHeader();
7     LogEntry* current = L;
8     do {
9         displayLogEntryColumn(current);
10        current = current->next;
11    } while (current != L);
12 }
13 \end{lstlisting}
14
15 \textbf{Functionality}:
16 \begin{itemize}
17     \item Takes the head pointer as input.
18     \item Checks for an empty list.
19     \item Prints a header.
20     \item Traverses the list starting from the head and continues until it returns to
21         the head, printing each node's details.
22 \end{itemize}
23 \subsection{Getting the Number of Log Entries (Circular)}

```

```

24 \label{ssub:log_size_circular}
25 The \texttt{LogSize} function (shared) calculates and returns the total number of log
    entries in the circular linked list.
26
27 \begin{lstlisting}
28 int LogSize(LogEntry* L) {
29     bool v = IsLogsEmpty(L);
30
31     if (v) {
32         return 0;
33     }
34
35     int counter = 0;
36     LogEntry* current = L;
37     do {
38         counter++;
39         current = current->next;
40     } while (current != L);
41
42     return counter;
43 }
44 \end{lstlisting}
45
46 \textbf{Functionality}:
47 \begin{itemize}
48     \item Takes the head pointer as input.
49     \item Returns 0 if the list is empty.
50     \item Otherwise, it traverses the list starting from the head and counts nodes
        until it returns to the head.
51 \end{itemize}
52
53 \subsection{Deleting a Log Entry by ID (Circular)}
54 \label{ssub:del_log_by_id_circular}
55 The \texttt{DelLogEntryByID} function (shared) searches for a log entry with a specific
    ID and removes it from the circular linked list.
56
57 \begin{lstlisting}
58 void DelLogEntryByID(LogEntry** L, int ID) {
59     if (*L == NULL) {
60         printf("\nList is empty\n");
61         return;
62     }
63
64     LogEntry *current = *L, *prev = NULL;
65     LogEntry *last = *L;
66     while (last->next != *L) {
67         last = last->next;
68     }
69
70     // Find the node to delete
71     do {
72         if (current->id == ID) {
73             break;
74         }
75         prev = current;
76         current = current->next;
77     } while (current != *L);
78
79     if (current->id != ID) {
80         printf("\nID NOT found\n");
81         return;
82     }
83
84     // If only one node
85     if (current->next == *L && prev == NULL) {
86         free(*L);
87         *L = NULL;
88         return;
89     }
90
91     // If first node
92     if (current == *L) {
93         last->next = current->next;

```

```

94     *L = (*L)->next;
95 } else {
96     prev->next = current->next;
97 }
98
99     free(current);
100 }
101 \end{lstlisting}
102
103 \textbf{Functionality}:
104 \begin{itemize}
105     \item Takes the head pointer and target ID as input.
106     \item Handles an empty list.
107     \item Finds the last node to handle deletion of the head in a circular list.
108     \item Traverses the list to find the node with the target ID.
109     \item Handles deletion of a single node, the head node, and other nodes by
        updating the 'next' pointers to bypass the node to be deleted.
110     \item Frees the memory of the deleted node.
111     \item Prints a "NOT found" message if the ID is not in the list.
112 \end{itemize}
113
114 \subsection{Deleting the First Log Entry (Circular)}
115 \label{ssub:del_log_beginning_circular}
116 The \texttt{DelLogAtTheBeginning} function (shared) removes the first log entry from the
        circular linked list.
117
118 \begin{lstlisting}
119 void DelLogAtTheBeginning(LogEntry** L) {
120     bool v = IsLogsEmpty(*L);
121
122     if (v) {
123         printf("They Are empty\n");
124         return;
125     }
126
127     LogEntry* temp = *L;
128     if (temp->next == *L) { // Only one node
129         free(temp);
130         *L = NULL;
131     } else {
132         LogEntry* last = *L;
133         while (last->next != *L) {
134             last = last->next;
135         }
136         *L = (*L)->next;
137         last->next = *L;
138         free(temp);
139     }
140 }
141 \end{lstlisting}
142
143 \textbf{Functionality}:
144 \begin{itemize}
145     \item Takes the head pointer as input.
146     \item Checks for an empty list.
147     \item Handles the case of a single node list.
148     \item For lists with more than one node, it finds the last node, updates the last
        node's 'next' to the second node, updates the head to the second node, and frees the
        original head.
149 \end{itemize}
150
151 \subsection{Deleting the Last Log Entry (Circular)}
152 \label{ssub:del_log_end_circular}
153 The \texttt{DelLogAtTheEnd} function (shared) removes the last log entry from the
        circular linked list.
154
155 \begin{lstlisting}
156 void DelLogAtTheEnd(LogEntry** L) {
157     bool v = IsLogsEmpty(*L);
158
159     if (v) {
160         printf("They Are empty\n");
161         return;

```

```

162     }
163
164     LogEntry* temp = *L;
165     LogEntry* prev = NULL;
166
167     if (temp->next == *L) { // Only one node
168         free(temp);
169         *L = NULL;
170         return;
171     }
172
173     while (temp->next != *L) {
174         prev = temp;
175         temp = temp->next;
176     }
177
178     prev->next = *L;
179     free(temp);
180 }
181 \end{lstlisting}
182
183 \textbf{Functionality}:
184 \begin{itemize}
185     \item Takes the head pointer as input.
186     \item Checks for an empty list.
187     \item Handles the case of a single node list.
188     \item For lists with more than one node, it traverses to the last node while
189         keeping track of the previous node.
190     \item It updates the 'next' pointer of the previous node to point to the head,
191         effectively removing the last node, and then frees the last node.
192 \end{itemize}
193
194 \subsection{Deleting a Log Entry by Timestamp (Circular)}
195 \label{ssub:del_log_by_timestamp_circular}
196 The \texttt{DelLogByTimeStamp} function (shared) searches for a log entry by timestamp
197 and removes it from the circular list.
198
199 \begin{lstlisting}
200 bool isValidDateTime(int year, int month, int day, int hour, int minutes, int seconds) {
201     // Validate year (assuming reasonable range)
202     if (year < 1900 || year > 2100) return false;
203
204     // Validate month
205     if (month < 1 || month > 12) return false;
206
207     // Validate day
208     if (day < 1 || day > 31) return false;
209
210     // Check months with 30 days
211     if ((month == 4 || month == 6 || month == 9 || month == 11) && day > 30)
212         return false;
213     if (hour >= 24 || hour < 0)
214     {
215         return false;
216     }
217
218     // Check February (ignoring leap years for simplicity)
219     if (month == 2 && day > 28) return false;
220
221     // Validate minutes and seconds
222     if (minutes < 0 || minutes > 59) return false;
223     if (seconds < 0 || seconds > 59) return false;
224
225     return true;
226 }
227
228 void DelLogByTimeStamp(LogEntry** L) { // Note: This function name conflicts with the
229     bidirectional list version
230     if (*L == NULL) {
231         printf("Log list is empty!\n");
232         return;
233     }
234     int year, month, day, hour, minutes, seconds;

```

```

231 char tryAgain;
232 char temp [50];
233 printf("Enter the TimeStamp in this format: 2023-12-25 15:30:45 W. Central Africa
Standard Time\n");
234 do {
235     printf("Enter date and time (YYYY MM DD mm ss): ");
236     scanf("%d %d %d %d %d %d", &year, &month, &day,&hour, &minutes, &seconds);
237
238     // Clear input buffer
239     while(getchar() != '\n');
240
241     if (!isValidDateTime(year, month, day, hour, minutes, seconds)) {
242         printf("Invalid input! Please try again.\n");
243         continue;
244     }
245
246     printf("\nYou entered: %04d-%02d-%02d %02d:%02d:%02d\n",
247         year, month, day, hour, minutes, seconds);
248
249     printf("Do you want to enter another date? (y/n): ");
250     scanf(" %c", &tryAgain);
251
252     // Clear input buffer
253     while(getchar() != '\n');
254
255 } while (tryAgain == 'y' || tryAgain == 'Y');
256
257 sprintf(temp, "%04d-%02d-%02d %02d:%02d:%02d", year, month, day, hour, minutes, seconds
);
258
259 LogEntry *current = *L;
260 LogEntry *previous = NULL;
261
262 // Find the node to delete
263 do {
264     if (strstr(current->timestamp, temp) != NULL) {
265         break;
266     }
267     previous = current;
268     current = current->next;
269 } while (current != *L);
270
271 // If node not found
272 if (strstr(current->timestamp, temp) == NULL) {
273     printf("No log found with timestamp containing '%s'\n", temp);
274     return;
275 }
276
277 // If only one node
278 if (current->next == *L && previous == NULL) {
279     free(*L);
280     *L = NULL;
281     printf("Log with timestamp containing '%s' deleted.\n", temp);
282     return;
283 }
284
285 // If first node
286 if (current == *L) {
287     LogEntry* last = *L;
288     while(last->next != *L) last = last->next;
289     last->next = current->next;
290     *L = (*L)->next;
291 } else {
292     previous->next = current->next;
293 }
294
295 free(current);
296 printf("Log with timestamp containing '%s' deleted.\n", temp);
297 }

```

Functionality:

- ↳ Prompts the user for date and time components and validates the input. Formats the input

into a string for comparison. Traverses the circular list to find a log entry whose timestamp contains the formatted input string. Handles deletion of a single node, the head node, and other nodes by updating the 'next' pointers to bypass the node to be deleted, maintaining circularity. Frees the memory of the deleted node. Prints a "No log found" message if no match is found.

i

19.5 Sorting Logs by ID (Circular - Merge Sort)

The `SortLogsById` function (shared) sorts the circular linked list by ID using a Merge Sort algorithm. This involves breaking the circularity, recursively splitting the list, sorting the sublists, merging them, and finally making the merged list circular again.

```

1 LogEntry* merge2List(LogEntry* h1, int size1, LogEntry* h2, int size2) {
2     if (h1 == NULL && h2 == NULL) return NULL;
3
4     LogEntry* dummy = malloc(sizeof(LogEntry));
5     LogEntry* tail = dummy;
6     dummy->next = NULL;
7
8     int count1 = 0, count2 = 0;
9
10    while (count1 < size1 && count2 < size2 && h1 != NULL && h2 != NULL) {
11        if (h1->id > h2->id) {
12            tail->next = h2;
13            h2 = h2->next;
14            count2++;
15        } else {
16            tail->next = h1;
17            h1 = h1->next;
18            count1++;
19        }
20        tail = tail->next;
21    }
22
23    while (count1 < size1 && h1 != NULL) {
24        tail->next = h1;
25        h1 = h1->next;
26        tail = tail->next;
27        count1++;
28    }
29
30    while (count2 < size2 && h2 != NULL) {
31        tail->next = h2;
32        h2 = h2->next;
33        tail = tail->next;
34        count2++;
35    }
36
37    tail->next = NULL; // Break the circular reference for merging
38    LogEntry* result = dummy->next;
39    free(dummy);
40    return result;
41 }
42
43 LogEntry* SortLogsById(LogEntry* head, int size) {
44     if (size <= 1) {
45         return head;
46     }
47
48     int mid = size / 2, counter = 0;
49     LogEntry* p1 = head;
50     LogEntry* p2 = NULL;
51     LogEntry* t = NULL;
52
53     // Split the list
54     while (counter < mid - 1) {
55         p1 = p1->next;
56         counter++;
57     }
58     p2 = p1->next;

```

```

59 p1->next = head; // Maintain circularity of first half (temporarily)
60 p1 = p2;
61
62 // Find the end of second half
63 counter = 0;
64 while (counter < size - mid - 1) {
65     p1 = p1->next;
66     counter++;
67 }
68 p1->next = p2; // Maintain circularity of second half (temporarily)
69
70 LogEntry* r1 = SortLogsById(head, mid);
71 LogEntry* r2 = SortLogsById(p2, size - mid);
72
73 // Break circularity for merging
74 LogEntry* temp = r1;
75 for (int i = 0; i < mid - 1; i++) {
76     temp = temp->next;
77 }
78 temp->next = NULL;
79
80 temp = r2;
81 for (int i = 0; i < size - mid - 1; i++) {
82     temp = temp->next;
83 }
84 temp->next = NULL;
85
86 head = merge2List(r1, mid, r2, size - mid);
87
88 // Make the merged list circular again
89 if (head != NULL) {
90     temp = head;
91     while (temp->next != NULL) {
92         temp = temp->next;
93     }
94     temp->next = head;
95 }
96
97 return head;
98 }

```

Functionality:

SortLogsById recursively splits the circular list into two halves by temporarily breaking the circularity. It recursively sorts the two halves. merge2List merges the two sorted linear lists into a single sorted linear list. After the merge, SortLogsById makes the resulting list circular again by connecting the last node's 'next' pointer to the head.

↳

19.6 Sorting Logs by Severity (Circular)

The SortBySeverity function (shared) sorts the circular linked list by severity using a distribution sort approach.

```

1 void SortBySeverity(LogEntry** head) {
2     if (*head == NULL) return;
3
4     LogEntry *info_head = NULL, *info_tail = NULL;
5     LogEntry *warn_head = NULL, *warn_tail = NULL;
6     LogEntry *error_head = NULL, *error_tail = NULL;
7
8     LogEntry* current = *head;
9     LogEntry* next = NULL;
10
11     do {
12         next = current->next;
13         current->next = current; // Temporarily make it self-circular
14
15         if (strcmp(current->severity, "INFO") == 0) {
16             if (info_head == NULL) {
17                 info_head = current;

```

```

18         info_tail = current;
19     } else {
20         info_tail->next = current;
21         info_tail = current;
22     }
23 }
24 else if (strcmp(current->severity, "WARNING") == 0) {
25     if (warn_head == NULL) {
26         warn_head = current;
27         warn_tail = current;
28     } else {
29         warn_tail->next = current;
30         warn_tail = current;
31     }
32 }
33 else { // ERROR
34     if (error_head == NULL) {
35         error_head = current;
36         error_tail = current;
37     } else {
38         error_tail->next = current;
39         error_tail = current;
40     }
41 }
42
43     current = next;
44 } while (current != *head);
45
46 *head = NULL;
47 LogEntry* new_tail = NULL;
48
49 if (info_head) {
50     *head = info_head;
51     new_tail = info_tail;
52 }
53 if (warn_head) {
54     if (*head == NULL) {
55         *head = warn_head;
56     } else {
57         new_tail->next = warn_head;
58     }
59     new_tail = warn_tail;
60 }
61 if (error_head) {
62     if (*head == NULL) {
63         *head = error_head;
64     } else {
65         new_tail->next = error_head;
66     }
67     new_tail = error_tail;
68 }
69
70 if (new_tail) {
71     new_tail->next = *head; // Make it circular
72 }
73 }

```

Functionality:

↳ Traverses the circular list, detaching each node by temporarily making it self-circular. Distributes nodes into three separate linear lists based on severity. Concatenates the three linear lists in the order INFO, then WARNING, then ERROR. Finally, makes the combined list circular by connecting the last node's 'next' pointer to the new head.

↳

19.7 Reversing the Circular Linked List

The `ReverseLogs` function (shared) reverses the order of the nodes in the circular linked list.

```

1 void ReverseLogs(LogEntry** H) {
2     if (IsLogsEmpty(*H) || (*H)->next == *H) {

```

```

3     return;
4 }
5
6 LogEntry* prev = *H;
7 LogEntry* current = (*H)->next;
8 LogEntry* next = NULL;
9
10 while (current != *H) {
11     next = current->next;
12     current->next = prev;
13     prev = current;
14     current = next;
15 }
16
17 (*H)->next = prev;
18 *H = prev;
19 }

```

↳

Functionality:

↳ Handles empty or single-node lists. Uses three pointers ('prev', 'current', 'next') to traverse the list. For each node, it reverses the 'next' pointer to point to the previous node. After the loop, the 'next' pointer of the original head (which is now the last node) is set to the new head ('prev'), completing the reversal and maintaining circularity. The head pointer is updated to the original last node.

↳

19.8 Searching for a Log Entry by ID (Circular)

The `serchLogById` function (shared) searches for a log entry with a specific ID in the circular linked list.

```

1 LogEntry* serchLogById(LogEntry* L, int ID) {
2     if (L == NULL) {
3         printf("it is empty\n");
4         return NULL;
5     }
6
7     LogEntry* current = L;
8     do {
9         if (current->id == ID) {
10            printf("it was found\n");
11            printf("%-5d | %-32s | %-10s | %-100s\n", current->id, current->timestamp,
12                current->severity, current->message);
13            return current;
14        }
15        current = current->next;
16    } while (current != L);
17
18    printf("Not found \n");
19    getchar();
20    return NULL;
21 }

```

↳

Functionality:

Takes the head pointer and target ID as input. Handles an empty list. Traverses the circular list starting from the head and continues until it returns to the head. Compares the ID of each node with the target ID. Prints details and returns the node if found, otherwise prints "Not found" and returns 'NULL'. **19.9 Generating Multiple Log Entries (Circular)**

The `GenLog` function (shared) allows the user to generate multiple log entries and add them to the circular list using `InsertLogEntryByPos`.

```

1 void GenLog(LogEntry** L, int size) {
2     int i;
3     for (i = 0; i < size; i++) {

```

```

4   InsertLogEntryByPos(L); // Inserts at a user-specified position
5   }
6   DisplayLogs(*L); % Display logs after generation
7 }

```

• **Functionality:**

- – Takes the head pointer and the number of logs to generate as input.
- Loops ‘size’ times, calling `InsertLogEntryByPos` to create and insert each new log entry at a position specified by the user, maintaining circularity.
- After generation, it displays the logs.

•

20 Algorithmic Complexity Analysis (Circular Linked Lists)

Let n be the number of log entries (nodes) in the circular linked list. The following is a complexity analysis for the core operations discussed in this chapter.

Time Complexity

Time complexity measures how the execution time of an algorithm grows with the input size.

`CreateLogEntry`: $O(1)$ `IsLogsEmpty`: $O(1)$ `AddLogEntryAtTheBeginning`: $O(n)$ - Requires traversing to the last node. `AddLogEntryAtTheEnd`: $O(n)$ - Requires traversing to the last node. `InsertLogEntry` (and `InsertLogEntryByPos`): $O(n)$ - Requires traversing up to the insertion position. `DisplayLogs`: $O(n)$ - Requires traversing the entire list. `LogSize`: $O(n)$ - Requires traversing the entire list. `DelLogEntryById`: $O(n)$ - Requires traversing up to the node to be deleted. `DelLogAtTheBeginning`: $O(n)$ - Requires traversing to the last node to update its ‘next’ pointer. `DelLogAtTheEnd`: $O(n)$ - Requires traversing to the second-to-last node. `DelLogByTimeStamp`: $O(n \times L)$ - Traversing the list ($O(n)$) and performing a substring search (`strstr`) within each timestamp string ($O(L)$). `SortLogsById` (Merge Sort): $O(n \log n)$ - Similar to linear lists, splitting, merging, and re-circularizing add overhead but the dominant factor is the merge sort. `SortBySeverity`: $O(n)$ - Traverses the list once for distribution and then concatenates. `ReverseLogs`: $O(n)$ - Requires traversing the entire list. `SearchLogById`: $O(n)$ - Requires traversing up to the node to be searched. `GenLog`: $O(\text{size} \times n)$ - Calls `InsertLogEntryByPos` ‘size’ times, which is $O(n)$ in the worst case.

Space Complexity

Space complexity measures the amount of memory used by an algorithm.

For most operations: $O(1)$ - Constant extra space for pointers and temporary variables. `SortLogsById` (Merge Sort): $O(\log n)$ for the recursion stack space. `SortBySeverity`: $O(1)$ - Uses a constant number of extra pointers. `GenLog`: $O(1)$ - Auxiliary space is constant, though the list itself grows.

Summary of Circular Linked List Complexities:

Table 3: Circular Linked List Operation Complexities

Operation	Time Complexity (Worst Case)	Space Complexity
CreateLogEntry	$O(1)$	$O(1)$
IsLogsEmpty	$O(1)$	$O(1)$
AddLogEntryAtTheBeginning	$O(n)$	$O(1)$
AddLogEntryAtTheEnd	$O(n)$	$O(1)$
InsertLogEntry	$O(n)$	$O(1)$
DisplayLogs	$O(n)$	$O(1)$
LogSize	$O(n)$	$O(1)$
DelLogEntryById	$O(n)$	$O(1)$
DelLogAtTheBeginning	$O(n)$	$O(1)$
DelLogAtTheEnd	$O(n)$	$O(1)$
DelLogByTimeStamp	$O(n \times L)$	$O(1)$
SortLogsById (Merge Sort)	$O(n \log n)$	$O(\log n)$
SortBySeverity	$O(n)$	$O(1)$
ReverseLogs	$O(n)$	$O(1)$
serchLogById	$O(n)$	$O(1)$
GenLog	$O(size \times n)$	$O(1)$

Queues

21 Introduction

A queue is a linear data structure that follows the First-In, First-Out (FIFO) principle. This means that the element that is added first to the queue will be the first one to be removed. Think of a queue like a line of people waiting; the person who arrived first is the first one to be served.

Queues are commonly used in computer science for managing tasks, handling requests, and in various algorithms where the order of processing is important. In the context of a log management system, a queue could be used to temporarily store incoming log entries before they are processed or stored in a more permanent structure like a linked list or database.

The implementation provided in ‘queue.c’ uses a fixed-size array to represent the queue, employing a circular buffer approach to efficiently manage the head and tail indices.

22 Log Entry Structure (Queue)

The ‘queue.c’ file defines a ‘LogEntry’ structure to hold the log data within the queue. Note that this structure is simpler than the linked list versions as it doesn’t contain ‘next’ or ‘prev’ pointers because the queue is implemented using an array.

```

1 typedef struct {
2     char message[MAX_LENGTH];
3     int level;
4 } LogEntry;

```

Structure Breakdown:

‘message’: A character array of size ‘MAX_LENGTH’ (defined as 256) to store the log message content. ‘level’: An integer to store the severity level of the log. Unlike the linked list versions, this uses an integer, which might correspond to a severity level.

23 Queue Structure

The queue itself is represented by the ‘LogQueue’ structure, which contains an array of ‘LogEntry’ and variables to manage the queue’s state.

```

1 #define MAX_SIZE 100
2 #define MAX_LENGTH 256
3
4 typedef struct {
5     LogEntry log[MAX_SIZE];
6     int head;
7     int tail;
8     int size;
9 }LogQueue;

```

Structure Breakdown:

‘log’: A fixed-size array of ‘LogEntry’ structures with a maximum capacity of ‘MAX_SIZE’ (defined as 100). This array is an integer index pointing to the front of the queue (the next element to be dequeued). ‘tail’: An integer index pointing to the current number of elements in the queue. The ‘head’ and ‘tail’ indices are managed using the modulo operator (‘

24 Queue Implementation Details

24.1 Initializing the Queue

The ‘initializequeue’ function sets the initial state of the ‘LogQueue’.

```

1 void initializequeue(LogQueue *p){
2     p->head=0;
3     p->tail=-1;
4     p->size=0;
5 }

```

Functionality:

Takes a pointer to a ‘LogQueue’ structure as input. Sets the ‘head’ index to 0. Sets the ‘tail’ index to -1. Sets the ‘size’ to 0, indicating an empty queue.

Note: The provided code snippet for ‘initializequeue’ in ‘queue.c’ has a parameter ‘LogEntry *q’ but uses ‘p-’. Assuming ‘p’ is the intended parameter name for the ‘LogQueue’ pointer.

24.2 Enqueuing a Log Entry

The ‘enqueue’ function adds a new log entry to the back of the queue.

```

1 int enqueue(LogQueue *p, const char *message, int level){
2     if (p->size==MAX_SIZE)
3     {
4         printf("U cannot add new elements because it is full.\n");
5         return 0; // Indicate failure
6     }
7     p->tail = (p->tail + 1) % MAX_SIZE;
8     strncpy(p->log[p->tail].message, message, MAX_LENGTH - 1);
9     p->log[p->tail].message[MAX_LENGTH - 1] = '\0';
10    p->log[p->tail].level = level;
11    p->size++;
12    return 1; // Indicate success
13 }

```

Functionality:

Takes a pointer to a ‘LogQueue’, a message string, and a level integer as input. Checks if the queue is full (‘p->size == MAX_SIZE’). If full, it prints an error and returns 0. If not full, it increments the ‘tail’ index around. Copies the provided message and level into the ‘LogEntry’ at the new ‘tail’ index. ‘strncpy’ is used for a safe string copy.

Note: The provided code snippet for ‘enqueue’ in ‘queue.c’ has a parameter ‘LogEntry *p’ but operates on a ‘LogQueue’. Assuming ‘LogQueue *p’ is the intended parameter type.

24.3 Dequeueing a Log Entry

The ‘dequeue’ function removes and returns the log entry from the front of the queue.

```
1 int dequeue(LogQueue *p, LogEntry *entry) {
2     if (p->size == 0) {
3         printf("Cannot dequeue because it is empty.\n");
4         return 0;
5     }
6     *entry = p->log[p->head];
7     p->head = (p->head + 1) % MAX_SIZE;
8     p->size--;
9
10    return 1;
11 }
```

ℓ

Functionality:

ℓ Takes a pointer to a ‘LogQueue’ and a pointer to a ‘LogEntry’ (to store the dequeued element) as input. Checks if the queue is empty (‘p->size == 0’). If empty, it prints an error and returns 0. If not empty, it copies the ‘LogEntry’ at the ‘head’ index to the provided ‘entry’ pointer. Increments the ‘head’ index using the modulo operator to handle wrap-around. Decrements the queue’s ‘size’. Returns 1 to indicate successful dequeuing.

ℓ

24.4 Peeking at the Front Entry

The ‘peek’ function allows viewing the log entry at the front of the queue without removing it.

```
1 bool peek(LogQueue *p, LogEntry *etr){
2     if (p->size == 0) {
3         printf("there is nothing to peek because it is empty.\n");
4         return false;
5     }
6     *etr = p->log[p->head];
7     return true;
8 }
```

ℓ

Functionality:

ℓ Takes a pointer to a ‘LogQueue’ and a pointer to a ‘LogEntry’ (to store the peeked element) as input. Checks if the queue is empty. If empty, it prints a message and returns ‘false’. If not empty, it copies the ‘LogEntry’ at the ‘head’ index to the provided ‘etr’ pointer. Returns ‘true’ to indicate success.

ℓ

24.5 Checking if the Queue is Empty

The ‘isEmpty’ function checks if the queue currently contains any elements.

```
1 bool isEmpty(LogQueue *p ){
2     if (p->size == 0) {
3         printf("Queue is empty.\n");
4         return true;
5     }
6     else{
7         return false;
8     }
9 }
```


↳

Functionality:

↳ Takes a pointer to a 'LogQueue' as input. Checks if the 'size' is 0. Prints "Queue is empty." and returns 'true' if the size is 0, otherwise returns 'false'.

↳

24.6 Checking if the Queue is Full

The 'isFull' function checks if the queue has reached its maximum capacity.

```
1 bool isFull(LogQueue *p ){
2     if (p->size == MAX_SIZE) {
3         printf("Queue is full.\n");
4         return true;
5     }
6     else{
7         return false;
8     }
9 }
```

↳

Functionality:

↳ Takes a pointer to a 'LogQueue' as input. Checks if the 'size' is equal to 'MAX_SIZE'. Prints "Queue is full." and returns 'true' if the size is equal to 'MAX_SIZE', otherwise returns 'false'.

25 Algorithmic Complexity Analysis (Queues)

Let N be the maximum capacity of the queue ('MAX_SIZE') and n be the current number of elements in the queue. The queue is implemented using an array and a circular buffer approach.

Time Complexity

Time complexity measures how the execution time of an algorithm grows with the input size.

initializequeue: $O(1)$ - Constant time to set initial values. **enqueue:** $O(1)$ - Constant time to update tail, copy data (assuming fixed 'MAX_LENGTH'), and increment size. **dequeue:** $O(1)$ - Constant time to copy data, update head, and decrement size.

Space Complexity

Space complexity measures the amount of memory used by an algorithm.

For all operations: $O(N)$ - The space required is dominated by the fixed-size array 'log' within the 'LogQueue' structure, which has a size of 'MAX_SIZE'. The space does not grow with the number of elements currently in the queue.

Summary of Queue Operation Complexities:

Table 4: Queue Operation Complexities (Array-based)

Operation	Time Complexity (Worst Case)	Space Complexity
initializequeue	$O(1)$	$O(N)$
enqueue	$O(1)$	$O(N)$
dequeue	$O(1)$	$O(N)$
peek	$O(1)$	$O(N)$
isEmpty	$O(1)$	$O(N)$
isFull	$O(1)$	$O(N)$

Stacks

26 Introduction

A stack is a linear data structure that follows the Last-In, First-Out (LIFO) principle. This means that the element that is added last to the stack will be the first one to be removed. Think of a stack like a pile of plates; you can only add or remove plates from the top of the pile.

Stacks are widely used in computer science for managing function calls (the call stack), parsing expressions, backtracking algorithms, and managing temporary data where the most recently added item is the next one needed. In the context of a log management system, a stack could potentially be used for operations requiring reversal of log order or temporary storage with LIFO access.

The implementation provided in ‘stacks.c’ uses a fixed-size array to represent the stack, with a ‘top’ index to keep track of the most recently added element.

27 Log Entry Structure (Stack)

The ‘stacks.c’ file defines a ‘LogEntry’ structure to hold the log data within the stack. Similar to the queue implementation, this structure is simpler than the linked list versions as it doesn’t contain pointers to other nodes.

```
1 typedef struct {
2     char message[MAX_LENGTH];
3     int level;
4 } LogEntry;
```

Structure Breakdown:

‘message’: A character array of size ‘MAX_LENGTH’ (defined as 256) to store the log message content. ‘level’: An integer to store the severity level of the log. This likely corresponds to defined severity levels (e.g., 1 for lowest, 10 for highest).

28 Stack Structure

The stack itself is represented by the ‘LogStack’ structure, which contains an array of ‘LogEntry’ and an index to manage the top of the stack.

```
1 #define MAX_SIZE 100
2 #define MAX_LENGTH 256
3
4 typedef struct {
5     LogEntry log[MAX_SIZE];
6     int top;
7 } LogStack;
```

Structure Breakdown:

‘log’: A fixed-size array of ‘LogEntry’ structures with a maximum capacity of ‘MAX_SIZE’ (defined as 100). This array holds an integer index pointing to the index of the most recently added element in the ‘log’ array. An empty stack is typically represented by -1.

29 Stack Implementation Details

29.1 Initializing the Stack

The ‘initializeStack’ function sets the initial state of the ‘LogStack’.

```
1 void initializeStack(LogStack *s) {
2     s->top = -1;
3 }
```

⌋

Functionality:

⌋ Takes a pointer to a 'LogStack' structure as input. Sets the 'top' index to -1, indicating that the stack is empty.

⌋

29.2 Checking if the Stack is Empty

The 'isStackEmpty' function checks if the stack currently contains any elements.

```
1 bool isStackEmpty(LogStack *s) {  
2     return (s->top == -1);  
3 }
```

⌋

Functionality:

⌋ Takes a pointer to a 'LogStack' as input. Returns 'true' if the 'top' index is -1, indicating an empty stack, and 'false' otherwise.

⌋

29.3 Checking if the Stack is Full

The 'isStackFull' function checks if the stack has reached its maximum capacity.

```
1 bool isStackFull(LogStack *s) {  
2     return (s->top == MAX_SIZE - 1);  
3 }
```

⌋

Functionality:

⌋ Takes a pointer to a 'LogStack' as input. Returns 'true' if the 'top' index is equal to 'MAX_SIZE - 1', indicating a full stack, and 'false' otherwise.⌋

29.4 Pushing a Log Entry onto the Stack

The 'pushLog' function adds a new log entry to the top of the stack.

```
1 bool pushLog(LogStack *s, const char *message, int level) {  
2     if (isStackFull(s)) {  
3         printf("Cannot push because stack is full.\n");  
4         return false;  
5     }  
6     s->top++;  
7     strncpy(s->log[s->top].message, message, MAX_LENGTH);  
8     s->log[s->top].message[MAX_LENGTH - 1] = '\0'; // Ensure null-termination  
9     s->log[s->top].level = level;  
10    return true;  
11 }
```

⌋

Functionality:

29.5 Popping a Log Entry from the Stack

The ‘popLog’ function removes and returns the log entry from the top of the stack.

```
1 bool popLog(LogStack *s, LogEntry *removed) {
2     if (isStackEmpty(s)) {
3         printf(" Nothing to pop because stack is empty.\n");
4         return false;
5     }
6     *removed = s->log[s->top];
7     s->top--;
8     return true;
9 }
```

ℓ

Functionality:

29.6 Peeking at the Top Entry

The ‘peekLog’ function allows viewing the log entry at the top of the stack without removing it.

```
1 bool peekLog(LogStack *s, LogEntry *top) {
2     if (isStackEmpty(s)) {
3         printf("There is nothing to peek.\n");
4         return false;
5     }
6     *top = s->log[s->top];
7     return true;
8 }
```

ℓ

Functionality:

29.7 Checking Stack State

The ‘checkStackState’ function provides a string indicating whether the stack is empty, full, or has space.

```
1 const char* checkStackState(LogStack *s) {
2     if (isStackEmpty(s)) {
3         return "empty";
4     }
5     if (isStackFull(s)){
6         return "full";
7     }
8     return "there is space";
9 }
```

ℓ

Functionality:

29.8 Inserting at the Bottom (Recursive Helper)

The ‘insertAtBottom’ function is a recursive helper function used by ‘reverseStack’ to insert an element at the bottom of the stack.

```
1 static void insertAtBottom(LogStack *s, LogEntry item) {
2     if (isStackEmpty(s)) {
3         pushLog(s, item.message, item.level);
4     } else {
5         LogEntry temp;
6         popLog(s, &temp);
7         insertAtBottom(s, item);
8         pushLog(s, temp.message, temp.level);
9     }
10 }
```

↳

Functionality:

↳ Takes a pointer to a 'LogStack' and a 'LogEntry' to insert as input. Base Case: If the stack is empty, it pushes the item directly. Recursive Step: If the stack is not empty, it pops the top element, recursively calls 'insertAtBottom' to insert the new item at the bottom of the remaining stack, and then pushes the popped element back onto the stack.

↳ Note: This implementation involves copying the 'LogEntry' data during the recursive calls. For larger data, this could be inefficient.

29.9 Reversing the Stack

The 'reverseStack' function reverses the order of elements in the stack using the 'insertAtBottom' helper function.

```
1 void reverseStack(LogStack *s) {  
2     if (!isStackEmpty(s)) {  
3         LogEntry item;  
4         popLog(s, &item);  
5         reverseStack(s);  
6         insertAtBottom(s, item);  
7     }  
8 }
```

Functionality:

↳ Takes a pointer to a 'LogStack' as input. Base Case: If the stack is empty, the recursion stops. Recursive Step: If the stack is not empty, it pops the top element, recursively calls 'reverseStack' to reverse the remaining stack, and then calls 'insertAtBottom' to place the popped element at the bottom of the now-reversed smaller stack.

↳

29.10 Displaying a Single Log Entry

The 'displayLog' function is a helper to print the details of a single 'LogEntry'.

```
1 void displayLog(LogEntry *entry) {  
2     printf("Level %d: %s\n", entry->level, entry->message);  
3 }
```

↳

Functionality:

30 Algorithmic Complexity Analysis (Stacks)

Let N be the maximum capacity of the stack ('MAX_SIZE') and n be the current number of elements in the stack. The stack implementation uses an array.

Time Complexity

Time complexity measures how the execution time of an algorithm grows with the input size.

initializeStack: $O(1)$ - Constant time to set the initial 'top' value. **isStackEmpty:** $O(1)$ - Constant time to check the 'top' index. **isStackFull:** $O(1)$ - Constant time to check the 'top' index against 'MAX_SIZE - 1'. **pushLog:** $O(1)$ - Constant time to increment 'top', copy data (assuming fixed 'MAX_LENGTH'). **popLog:** $O(1)$ - Constant time to copy data and decrement 'top'. **reverseStack:** - It calls 'popLog' n times and 'insertAtBottom' n times. 'insertAtBottom' is $O(n)$, leading to an overall $O(n \times n) = O(n^2)$ complexity. **displayLog:** $O(1)$ - Constant time to print a single entry.

Space Complexity

Space complexity measures the amount of memory used by an algorithm.

For most operations: $O(N)$ - The space required is dominated by the fixed-size array 'log' within the 'LogStack' structure, which has a size of 'MAX_SIZE'. *The space does not grow with the number of elements.*

31 Comparison of Queues and Stacks

Queues and stacks are both fundamental linear data structures used for storing and managing collections of elements. While they share the characteristic of organizing data in a sequence, their primary difference lies in the order in which elements are accessed and removed. This difference dictates their typical use cases and the implementation of their core operations.

31.1 Fundamental Principles

The core distinction between queues and stacks is their access principle:

- **Queue:** Follows the **First-In, First-Out (FIFO)** principle. The element that was added first is the first one to be removed. Elements are added at one end (the "rear" or "tail") and removed from the other end (the "front" or "head").
- **Stack:** Follows the **Last-In, First-Out (LIFO)** principle. The element that was added last is the first one to be removed. Elements are added and removed from the same end (the "top").

31.2 Core Operations

The primary operations on queues and stacks reflect their access principles:

- **Queue Operations** (as seen in Chapter 20):
 - * **enqueue:** Adds an element to the rear of the queue.
 - * **dequeue:** Removes an element from the front of the queue.
 - * **peek:** Views the element at the front without removing it.
 - * **isEmpty, isFull:** Check the state of the queue.
- **Stack Operations** (as seen in Chapter 25):
 - * **pushLog:** Adds an element to the top of the stack.
 - * **popLog:** Removes an element from the top of the stack.
 - * **peekLog:** Views the element at the top without removing it.
 - * **isEmpty, isStackFull:** Check the state of the stack.

As discussed in the complexity analysis sections (Section 25 and 30), the core enqueue/dequeue and push/pop/peek operations typically have a time complexity of $O(1)$ for both array-based and linked-list based implementations (though linked list implementations might have $O(n)$ for operations like adding/removing at the end in a singly linked list).

31.3 Typical Use Cases

The distinct access methods make queues and stacks suitable for different applications:

- **Queue Use Cases:**
 - * Managing requests in a shared resource (e.g., printer queue, CPU scheduling).
 - * Breadth-First Search (BFS) algorithm.
 - * Handling asynchronous data transfer (buffering).
 - * Call center systems (handling calls in order).
- **Stack Use Cases:**
 - * Function call management (the program's call stack).
 - * Expression evaluation and parsing (e.g., converting infix to postfix).

- * Backtracking algorithms (e.g., solving mazes, Sudoku).
- * Undo/Redo functionality in software.
- * Depth-First Search (DFS) algorithm.

31.4 Summary Table

The table below summarizes the key differences between queues and stacks.

Table 6: Comparison of Queues and Stacks		
Feature	Queue	Stack
Principle	FIFO (First-In, First-Out)	LIFO (Last-In, First-Out)
Insertion End	Rear (Tail)	Top
Removal End	Front (Head)	Top
Primary Insertion Operation	Enqueue	Push
Primary Removal Operation	Dequeue	Pop
Analogy	Line of people, pipe	Pile of plates, stack of books

In conclusion, while both queues and stacks are linear data structures, their fundamental access principles lead to different operational behaviors and make them appropriate tools for distinct computational problems. The choice between using a queue or a stack depends entirely on the specific requirements of the application and the desired order of element processing.

Recursion

32 Introduction to Recursion

Recursion is a programming technique where a function calls itself in order to solve a problem. A recursive function breaks down a complex problem into smaller, identical subproblems until it reaches a simple base case that can be solved directly. The solutions to the subproblems are then combined to solve the original problem.

Key components of a recursive function are:

- **Base Case:** A condition that stops the recursion. Without a base case, the function would call itself indefinitely, leading to a stack overflow.
- **Recursive Step:** The part of the function where it calls itself with a modified input, moving closer to the base case.

Recursion can often provide elegant and concise solutions for problems that have a naturally recursive structure, such as tree traversals, graph algorithms, and certain mathematical calculations.

33 Recursive Functions in the Log Management System

The ‘recursion.c’ file demonstrates the use of recursion for various tasks, including mathematical calculations and linked list operations.

33.1 Factorial Calculation

The ‘factorial’ function calculates the factorial of a non-negative integer recursively.

```

1 int factorial(int n){
2     if (n<0)
3     {
4         return -1; // Handle negative input
5     }
6     if (n==0)

```

```

7   {
8       return 1; // Base case: factorial of 0 is 1
9   }
10  return n*factorial(n-1); // Recursive step
11 }

```

Functionality:

- **Base Case:** If n is 0, it returns 1.
- **Recursive Step:** If $n > 0$, it returns n multiplied by the factorial of $n - 1$.
- Handles negative input by returning -1.

33.2 Fibonacci Series Calculation

The ‘fibonacciSeries’ function calculates the n -th term of the Fibonacci series recursively. The Fibonacci series starts with 0 and 1, and each subsequent term is the sum of the two preceding ones (e.g., 0, 1, 1, 2, 3, 5, 8...).

```

1  int fibonacciSeries(int n){
2      if (n<0)
3      {
4          return -1; // Handle negative input
5      }
6
7      if (n==0)
8      {
9          return 0; // Base case: 0th term is 0
10     }
11     if (n==1)
12     {
13         return 1; // Base case: 1st term is 1
14     }
15     return fibonacciSeries(n-1)+fibonacciSeries(n-2); // Recursive step
16 }

```

↳

Functionality:

33.3 Recursive Linked List Reversal

The ‘recursion.c’ file includes functions to reverse a singly linked list using recursion. This involves a helper function ‘ReverseSubFunction’ and a main function ‘ReverseWithRecursion’.

Helper Function: ‘ReverseSubFunction’

This function recursively reverses a linked list and helps update the new head of the reversed list.

```

1  LogEntry*ReverseSubFunction(LogEntry*L,LogEntry**New){
2      if (L->next==NULL){
3          *New=L; // Base case: The last node is the new head
4          return L;
5      }
6      ReverseSubFunction(L->next,New)->next=L; // Recursive step: Reverse the rest
          and link current node
7      L->next=NULL; // Set the next of the current node to NULL (it becomes the new
          tail of its sublist)
8      return L; // Return the current node
9  }

```

↳

Functionality:

- ↳ **Base Case:** If the current node ‘L’ is the last node (its ‘next’ is ‘NULL’), it sets the ‘*New’ pointer (which points to the head pointer in ‘ReverseWithRecursion’) to this node and returns

the node. **Recursive Step:** It recursively calls ‘ReverseSubFunction’ on the rest of the list (‘L->next’). The recursive call returns the *previous* node in the original list (which is the next node in the reversed list). It then sets the ‘next’ pointer of that returned node to the current node ‘L’, effectively linking ‘L’ into the reversed list. It sets ‘L->next’ to ‘NULL’ as ‘L’ will become the tail of the sublist being processed in this step.

↵

Main Function: ‘ReverseWithRecursion’

This function initiates the recursive reversal process.

```
1 void ReverseWithRecursion(LogEntry**L){
2     LogEntry*Temp; // Temporary pointer to hold the new head
3     ReverseSubFunction(*L,&Temp); // Call the recursive helper
4     *L=Temp; // Update the main head pointer
5 }
```

↵

Functionality:

↵ Takes a pointer to the head pointer of the list (‘LogEntry**L’). Declares a temporary pointer ‘Temp’. Calls the recursive helper ‘ReverseSubFunction’, passing the current head and the address of ‘Temp’. The helper will reverse the list and store the new head’s address in ‘Temp’. Updates the main head pointer ‘*L’ to the value stored in ‘Temp’, making the reversed list the new list.

↵

33.4 Finding Maximum Log ID Recursively

The ‘FindMAxLogId’ function finds the maximum ‘id’ among all log entries in a singly linked list using recursion.

```
1 int FindMAxLogId(LogEntry*L){
2     if (IsLogsEmpty(L))
3     {
4         return -1; // Base case: Empty list, return error value
5     }
6     if (L->next==NULL){
7         return L->id; // Base case: Single node, return its ID
8     }
9     // Recursive step: Compare current node's ID with max ID in the rest of the
10    list
11    return FindMAxLogId(L->next)>L->id ? FindMAxLogId(L->next) : L->id;
12 }
```

Functionality:

- **Base Cases:** If the list is empty, it returns -1. If the current node is the last node, it returns its ID.
- **Recursive Step:** It recursively calls ‘FindMAxLogId’ on the rest of the list (‘L->next’) to find the maximum ID in the sublist. It then compares this maximum ID from the sublist with the ID of the current node (‘L->id’) and returns the larger of the two.

34 Algorithmic Complexity Analysis (Recursion)

The complexity of recursive functions is analyzed based on the number of recursive calls and the work done in each call.

Time Complexity

Time complexity measures how the execution time of an algorithm grows with the input size.

- `factorial(n)`: $O(n)$ - The function makes n recursive calls, and each call involves constant time operations.
- `fibonacciSeries(n)`: $O(2^n)$ - This is a highly inefficient recursive implementation due to repeated calculations of the same Fibonacci numbers. The number of calls grows exponentially with n . An iterative approach or memoization can reduce this to $O(n)$.
- `ReverseSubFunction(L, New)` and `ReverseWithRecursion(L)`: $O(n)$ - The recursive helper visits each node in the linked list exactly once to reverse the pointers.
- `FindMaxLogId(L)`: $O(n)$ - The function visits each node in the linked list exactly once to compare IDs.

Space Complexity

Space complexity measures the amount of memory used by an algorithm.

- `factorial(n)`: $O(n)$ - Due to the recursive calls, the call stack can grow up to a depth of n .
- `fibonacciSeries(n)`: $O(n)$ - The depth of the recursion tree can go up to n .
- `ReverseSubFunction(L, New)` and `ReverseWithRecursion(L)`: $O(n)$ - The depth of the recursion is proportional to the number of nodes in the list.
- `FindMaxLogId(L)`: $O(n)$ - The depth of the recursion is proportional to the number of nodes in the list.

Summary of Recursive Function Complexities:

Table 7: Recursive Function Complexities

Function	Time Complexity (Worst Case)	Space Complexity
<code>factorial(n)</code>	$O(n)$	$O(n)$
<code>fibonacciSeries(n)</code>	$O(2^n)$	$O(n)$
<code>ReverseWithRecursion</code> (on a list of size n)	$O(n)$	$O(n)$
<code>FindMaxLogId</code> (on a list of size n)	$O(n)$	$O(n)$

Binary Search Trees

35 Introduction to Trees

Trees are non-linear data structures that organize data in a hierarchical manner. They consist of nodes connected by edges. A tree has a root node, and each node can have zero or more child nodes. Unlike linear structures like linked lists, trees allow for more efficient searching, insertion, and deletion in many cases, depending on the type of tree.

36 Introduction to Binary Search Trees

A Binary Search Tree (BST) is a special type of binary tree where for each node:

- All nodes in the left subtree have a key (value) less than the node's key.
- All nodes in the right subtree have a key (value) greater than the node's key.
- Both the left and right subtrees are also Binary Search Trees.

This ordering property makes BSTs particularly efficient for searching, as you can quickly eliminate half of the remaining tree at each step based on the comparison with the current node's key. The 'trees.c' implementation uses the timestamp of the log entry as the key for ordering nodes in the BST.

37 Log Entry Structure (Binary Search Tree)

The ‘trees.c’ file defines a specific structure for nodes in the Binary Search Tree, named ‘BsTLogEntry’. This structure includes pointers to both left and right children, in addition to the log entry data.

```
1 struct BsTLogEntry
2 {
3     struct BsTLogEntry *left;
4     int id;
5     char message[100];
6     struct BsTLogEntry *right;
7     char severity[10];
8     char timestamp[50];
9 };
10 typedef struct BsTLogEntry BsTLogEntry;
```

Structure Breakdown:

- ‘left’: A pointer to the left child node. According to BST properties, this child and its subtree will contain log entries with timestamps lexicographically less than the current node’s timestamp.
- ‘id’: An integer identifier for the log entry.
- ‘message’: A character array to store the log message content.
- ‘right’: A pointer to the right child node. This child and its subtree will contain log entries with timestamps lexicographically greater than or equal to the current node’s timestamp (based on the ‘ADDNode’ implementation’s comparison).
- ‘severity’: A character array for the log’s severity level.
- ‘timestamp’: A character array storing the log’s timestamp, used as the key for BST ordering.

38 Binary Search Tree Implementation Details

38.1 Creating a New BST Log Entry Node

The ‘CreateBsTLogEntry’ function allocates memory for a new ‘BsTLogEntry’ node and initializes its fields.

```
1 BsTLogEntry*CreateBsTLogEntry(){
2     BsTLogEntry* temp = (BsTLogEntry*)malloc(sizeof(BsTLogEntry));
3     if (temp == NULL) {
4         printf("\nMemory Allocation Failed\n");
5         return NULL;
6     }
7
8     temp->id = ID++; // Assuming ID is a global counter
9     temp->right = NULL;
10    temp->left = NULL; // Initialize child pointers to NULL
11    char msg[100];
12    char msg2[100];
13    char* temp3 = getCurrentTimeString(); // Function to get timestamp
14
15    if (temp3) {
16        strcpy(temp->timestamp, temp3);
17        free(temp3);
18    } else {
19        strcpy(temp->timestamp, "ERROR");
20    }
21
22    printf("\nWhat is the message?\n");
23    fgets(msg2, sizeof(msg2), stdin);
24    size_t len = strlen(msg2);
25    if (len > 0 && msg2[len - 1] == '\n') {
26        msg2[len - 1] = '\0';
27    }
28    strcpy(temp->message, msg2);
29
30    do {
```

```

31     printf("What is the level of severity (INFO, WARNING, ERROR)?\n");
32     if (fgets(msg, sizeof(msg), stdin) != NULL) {
33         size_t len = strlen(msg);
34         if (len > 0 && msg[len - 1] == '\n') {
35             msg[len - 1] = '\0';
36         }
37         strcpy(temp->severity, msg);
38     } else {
39         printf("Error reading severity level.\n");
40         free(temp);
41         return NULL;
42     }
43     while (strcmp(temp->severity, "INFO") != 0 && strcmp(temp->severity, "WARNING")
44            != 0 && strcmp(temp->severity, "ERROR") != 0);
45     return temp;
46 }

```

Functionality:

Allocates memory for a new 'BsTLogEntry' node using 'malloc'. Assigns a unique ID (assuming a global 'ID' counter). Initializes 'left' and 'right' child pointers to 'NULL'. Gets the current timestamp and copies it to the 'timestamp' field. Prompts the user for the log message and severity level, reading and validating input. Returns the newly created node or 'NULL' on memory allocation or input failure.

↳

38.2 Adding a Node to the BST (Recursive Helper)

The 'ADDNode' function is a recursive helper function used to find the correct position to insert a new node in the BST based on its timestamp.

```

1 void ADDNode(BsTLogEntry* B, BsTLogEntry* New) {
2     if (B == NULL)
3     {
4         return; // Should not happen if called correctly from AddLogEntryToBst
5     }
6
7     if (strcmp(New->timestamp, B->timestamp) >= 0) { // Compare timestamps
8         if (B->right == NULL) {
9             B->right = New; // Insert as right child
10            return;
11        } else {
12            ADDNode((B->right), New); // Recurse on the right subtree
13        }
14    } else {
15        if (B->left == NULL) {
16            B->left = New; // Insert as left child
17            return;
18        } else {
19            ADDNode((B->left), New); // Recurse on the left subtree
20        }
21    }
22 }

```

↳

Functionality:

Takes the current node 'B' and the new node 'New' to be added as input. Compares the timestamp of the 'New' node with the timestamp of the current node 'B' using 'strcmp'. If the new timestamp is greater than or equal to the current node's timestamp, it checks the right child. If the right child is 'NULL', it inserts the 'New' node there. Otherwise, it recursively calls 'ADDNode' on the right subtree. If the new timestamp is less than the current node's timestamp, it checks the left child. If the left child is 'NULL', it inserts the 'New' node there. Otherwise, it recursively calls 'ADDNode' on the left subtree.

↳

38.3 Adding a Log Entry to the BST

The ‘AddLogEntryToBst’ function is the main function for adding a new log entry node to the BST.

```
1 void AddLogEntryToBst (BsTLogEntry**root, BsTLogEntry*new){
2     if (*root==NULL)
3     {
4         *root=new; // If BST is empty, the new node becomes the root
5         return ;
6     }
7     ADDNode(*root,new); // Use the recursive helper to find insertion point
8 }
```

↳

Functionality:

↳ Takes a pointer to the root pointer of the BST (‘BsTLogEntry**root’) and the new node ‘new’ to be added as input. If the BST is currently empty (‘*root’ is ‘NULL’), the new node becomes the root. If the BST is not empty, it calls the ‘ADDNode’ helper function to recursively find the correct position for the new node and insert it.

↳

38.4 Displaying a Single BST Log Entry Node

The ‘displayBsTLogEntryColumn’ function is a helper to print the details of a single ‘BsTLogEntry’ node in a formatted way, similar to the linked list display.

```
1 void displayBsTLogEntryColumn(BsTLogEntry *entry) {
2     printf("%-5d | %-32s | %-10s | %-100s\n",
3           entry->id, entry->timestamp, entry->severity, entry->message);
4 }
```

↳

Functionality:

↳ Takes a pointer to a ‘BsTLogEntry’ node as input. Prints the ‘id’, ‘timestamp’, ‘severity’, and ‘message’ fields of the node in a columnar format.

↳

38.5 Finding the Minimum Node in a Subtree

The ‘findMinBstNode’ function finds the node with the minimum key (timestamp) in a given subtree. In a BST, the minimum key is always the leftmost node in the subtree.

```
1 BsTLogEntry* findMinBstNode(BsTLogEntry* node) {
2     BsTLogEntry* current = node;
3
4     while (current && current->left != NULL) {
5         current = current->left; // Traverse left until the leftmost node is found
6     }
7     return current; // Return the node with the minimum key
8 }
```

↳

Functionality:

↳ Takes a pointer to the root of the subtree (‘node’) as input. Starts from the given node and repeatedly moves to the left child as long as the left child is not ‘NULL’. Returns the pointer to the leftmost node in the subtree, which has the minimum timestamp.

↳

38.6 Deleting a Node from the BST (Recursive)

The 'DelBsTLogEntryNodeRecursive' function is a recursive helper function to delete a node with a specific timestamp from the BST.

```
1 BsTLogEntry* DelBsTLogEntryNodeRecursive(BsTLogEntry* root, char target_timestamp
2     []) {
3     if (root == NULL) {
4         printf("Node with timestamp '%s' not found.\n", target_timestamp);
5         return root; // Base case: Node not found
6     }
7
8     int cmp = strcmp(target_timestamp, root->timestamp);
9
10    // Traverse the tree to find the node to delete
11    if (cmp < 0) {
12        root->left = DelBsTLogEntryNodeRecursive(root->left, target_timestamp); //
13        Recurse left
14    }
15    else if (cmp > 0) {
16        root->right = DelBsTLogEntryNodeRecursive(root->right, target_timestamp);
17        // Recurse right
18    }
19
20    // Node with target_timestamp found
21    else {
22        // Case 1: Node with only one child or no child
23        if (root->left == NULL) {
24            BsTLogEntry* temp = root->right;
25            free(root); // Free the node
26            printf("Node with timestamp '%s' deleted.\n", target_timestamp);
27            return temp; // Return the right child (or NULL if no child)
28        } else if (root->right == NULL) {
29            BsTLogEntry* temp = root->left;
30            free(root); // Free the node
31            printf("Node with timestamp '%s' deleted.\n", target_timestamp);
32            return temp; // Return the left child
33        }
34
35        // Case 2: Node with two children
36        // Get the in-order successor (smallest in the right subtree)
37        BsTLogEntry* temp = findMinBstNode(root->right);
38
39        // Copy the in-order successor's data to this node
40        root->id = temp->id;
41        strncpy(root->message, temp->message, sizeof(root->message) - 1);
42        root->message[sizeof(root->message) - 1] = '\0';
43        strncpy(root->severity, temp->severity, sizeof(root->severity) - 1);
44        root->severity[sizeof(root->severity) - 1] = '\0';
45        strncpy(root->timestamp, temp->timestamp, sizeof(root->timestamp) - 1);
46        root->timestamp[sizeof(root->timestamp) - 1] = '\0';
47
48        // Delete the in-order successor from the right subtree
49        root->right = DelBsTLogEntryNodeRecursive(root->right, temp->timestamp);
50    }
51    return root; // Return the (potentially updated) root of the subtree
52 }
```

Functionality:

↳ Takes the current root of the subtree and the target timestamp as input. **Base Case:** If the current root is 'NULL', the node is not found, and it returns 'NULL'. It compares the target timestamp with the current node's timestamp. If the target is smaller, it recursively calls on the left subtree. If the target is larger, it recursively calls on the right subtree. If the target timestamp matches the current node's timestamp, the node to be deleted is found. It handles three deletion cases:

- * Node with no children or one child: The node is freed, and its child (or 'NULL') is returned to link with the parent.
- * Node with two children: The in-order successor (the smallest node in the right subtree) is found. The data from the in-order successor is copied to the current node. Then, the

in-order successor is recursively deleted from the right subtree.

- Prints a message indicating whether the node was found and deleted.
- Returns the root of the modified subtree.

↳

38.7 Deleting a Node from the BST

The ‘DelBsTLogEntryNode’ function is the main function to initiate the recursive deletion process from the BST.

```
1 void DelBsTLogEntryNode(BsTLogEntry** root, char target_timestamp[]) {
2     if (*root == NULL) {
3         printf("BST is empty. Cannot delete.\n");
4         return;
5     }
6     *root = DelBsTLogEntryNodeRecursive(*root, target_timestamp); // Call the
7     recursive helper
8 }
```

↳

Functionality:

↳ Takes a pointer to the root pointer of the BST and the target timestamp as input. Checks if the BST is empty. Calls the recursive helper function ‘DelBsTLogEntryNodeRecursive’ to perform the deletion, updating the root pointer if the root node is deleted.

↳

38.8 BST Traversal: In-Order

The ‘PrintBsTLogEntryInOrder’ function performs an in-order traversal of the BST. In-order traversal of a BST visits nodes in ascending order of their keys (timestamps).

```
1 void PrintBsTLogEntryInOrder(BsTLogEntry*B){
2     if (B==NULL)
3     {
4         return; // Base case: Empty subtree
5     }
6     PrintBsTLogEntryInOrder(B->left); // Traverse left subtree
7     displayBsTLogEntryColumn(B); // Visit current node
8     PrintBsTLogEntryInOrder(B->right); // Traverse right subtree
9 }
```

↳

Functionality:

↳ **Base Case:** If the current node ‘B’ is ‘NULL’, the recursion stops. **Recursive Step:** It recursively calls itself on the left subtree, then visits and prints the current node’s data, and finally recursively calls itself on the right subtree.

↳

38.9 BST Traversal: Pre-Order

The ‘PrintBsTLogEntryPreOrder’ function performs a pre-order traversal of the BST. In pre-order traversal, the current node is visited before its children.

```
1 void PrintBsTLogEntryPreOrder(BsTLogEntry*B){
2     if (B==NULL)
3     {
4         return; // Base case: Empty subtree
5     }
6     displayBsTLogEntryColumn(B); // Visit current node
7 }
```

```

7   PrintBsTLogEntryPreOrder(B->left); // Traverse left subtree
8   PrintBsTLogEntryPreOrder(B->right); // Traverse right subtree
9 }

```

↳

Functionality:

↳ **Base Case:** If the current node 'B' is 'NULL', the recursion stops. **Recursive Step:** It visits and prints the current node's data, then recursively calls itself on the left subtree, and finally recursively calls itself on the right subtree.

↳

38.10 BST Traversal: Post-Order

The 'PrintBsTLogEntryPostOrder' function performs a post-order traversal of the BST. In post-order traversal, the current node is visited after its children.

```

1  void PrintBsTLogEntryPostOrder(BsTLogEntry*B){
2      if (B==NULL)
3      {
4          return; // Base case: Empty subtree
5      }
6      PrintBsTLogEntryPostOrder(B->left); // Traverse left subtree
7      PrintBsTLogEntryPostOrder(B->right); // Traverse right subtree
8      displayBsTLogEntryColumn(B); // Visit current node
9  }
10 \end{lstlisting}
11
12 \textbf{Functionality}:
13 \begin{itemize}
14     \item \textbf{Base Case}: If the current node 'B' is 'NULL', the recursion
15     stops.
16     \item \textbf{Recursive Step}: It recursively calls itself on the left subtree,
17     then recursively calls itself on the right subtree, and finally visits and
18     prints the current node's data.
19 \end{itemize}
20
21 \subsection{Creating a BST Node from a LogEntry Copy}
22 \label{ssub:create_node_cpy_to_bst}
23 The 'CreateNodeCpyToBst' function creates a new 'BsTLogEntry' node and copies data
24 from an existing 'LogEntry' (from a linked list).
25
26 \begin{lstlisting}
27 BsTLogEntry*CreateNodeCpyToBst(LogEntry*original){
28     if (original==NULL)
29     {
30         return NULL; // Handle NULL input
31     }
32     BsTLogEntry*root=malloc(sizeof(BsTLogEntry));
33     if (root==NULL)
34     {
35         return root; // Handle memory allocation failure
36     }
37
38     root->id=original->id;
39     root->left=NULL;
40     root->right=NULL; // Initialize child pointers
41     strncpy(root->timestamp,original->timestamp,sizeof(root->timestamp)-1);
42     root->timestamp[sizeof(root->timestamp)-1]='\0'; // Ensure null-termination
43     strncpy(root->severity,original->severity,sizeof(root->severity)-1);
44     root->severity[sizeof(root->severity)-1]='\0'; // Ensure null-termination
45     strncpy(root->message,original->message,sizeof(root->message)-1);
46     root->message[sizeof(root->timestamp)-1]='\0'; // Ensure null-termination
47     return root ;
48 }

```

Functionality:

38.11 Converting a Linked List to a BST

The 'convertLogEntryToBst' function converts a singly linked list of 'LogEntry' nodes into a Binary Search Tree of 'BsTLogEntry' nodes, using the timestamp as the key.

```
1 BsTLogEntry*convertLogEntryToBst(LogEntry*H){
2     if (H==NULL)
3     {
4         return NULL; // Base case: Empty linked list
5     }
6     else
7     {
8         BsTLogEntry*root=NULL; // Initialize the root of the new BST
9         while (H!=NULL)
10        {
11            BsTLogEntry*New=CreateNodeCpyToBst(H); // Create a new BST node from
the current linked list node
12            if (New != NULL) {
13                AddLogEntryToBst(&root,New); // Add the new BST node to the BST
14            }
15            H=H->next; // Move to the next node in the linked list
16        }
17        return root; // Return the root of the constructed BST
18    }
19 }
```

↳

Functionality:

- ↳ Takes the head pointer of a singly linked list ('LogEntry*H') as input. **Base Case:** If the linked list is empty, it returns 'NULL'. Initializes the root of the new BST to 'NULL'. Traverses the linked list. For each 'LogEntry' in the linked list:
 - * It calls 'CreateNodeCpyToBst' to create a new 'BsTLogEntry' node with copied data.
 - * If the new BST node is created successfully, it calls 'AddLogEntryToBst' to insert this node into the BST.
- ↳
- Returns the root of the constructed Binary Search Tree.

↳

38.12 Queue Implementation for Level Order Traversal (Helper)

The 'trees.c' file includes a basic queue implementation ('struct node', 'Queue', 'createQueue', 'EnQueue', 'DeQueue', 'isEmptyQueue'). While a full level order traversal function is not explicitly shown using these, this queue structure and its operations are typically used to perform Level Order (Breadth-First) Traversal of a tree.

Node Structure for Queue

```
1 typedef struct node
2 {
3     BsTLogEntry*val;
4     struct node *next;
5 }node;
```

This structure defines a node for a linked list-based queue, where 'val' stores a pointer to a 'BsTLogEntry' (a BST node).

Queue Structure

```
1 typedef struct Queue{
2     node*front;
3     node*rear;
4 }Queue;
```

This structure represents the queue with pointers to the front and rear nodes.

Queue Operations

```
1 Queue*createQueue(){
2     Queue *Q =malloc(sizeof(Queue));
3     Q->front=NULL;
4     Q->rear=NULL;
5     return Q;
6 }
7 void EnQueue(Queue*Q,BsTLogEntry*value){
8     node *Temp=NULL;
9     if (value==NULL)
10    {
11        return;
12    }
13
14    Temp=malloc(sizeof(node));
15    if(Temp==NULL){
16        return ;
17    }
18    Temp->next=NULL;
19    Temp->val=value;
20    if(Q->front==NULL){
21        Q->front=Temp;
22        Q->rear=Temp;
23    }
24    else{
25        Q->rear->next=Temp;
26        Q->rear=Temp;
27    }
28 }
29 BsTLogEntry* DeQueue(Queue*Q){
30     if(Q->front==NULL){
31         return NULL;
32     }
33     else if (Q->front->next==NULL){
34         BsTLogEntry* val=Q->front->val;
35         free(Q->front);
36         Q->front=NULL;
37         Q->rear=NULL;
38         return val;
39     }
40     else{
41         BsTLogEntry *val=Q->front->val;
42         node * Temp =Q->front->next;
43         free(Q->front);
44         Q->front=Temp;
45         return val;
46     }
47 }
48 bool isEmptyQueue(Queue*Q){
49     return (Q->front==NULL&&Q->rear==NULL);
50 }
```

Functionality:

↳ ‘createQueue’: Allocates memory for and initializes an empty queue. ‘EnQueue’: Adds a ‘BsTLogEntry’ pointer to the rear of the queue. ‘DeQueue’: Removes and returns a ‘BsTLogEntry’ pointer from the front of the queue. ‘isEmptyQueue’: Checks if the queue is empty.

↳ These functions suggest the intention to implement level order traversal, which uses a queue to process nodes level by level.

38.13 Finding and Deleting the Minimum Node (Alternative/Helper)

The ‘findMinBstNodeAndDel’ function finds the node with the minimum key in a subtree and also removes it from the tree. This function seems to be an alternative approach or a helper function for deletion, specifically for finding and detaching the in-order successor.

```
1 BsTLogEntry* findMinBstNodeAndDel(BsTLogEntry* node) {
2     BsTLogEntry* current = node,*prev=NULL;
```

```

3
4     while (current && current->left != NULL) {
5         prev=current;
6         current = current->left; // Traverse left, keeping track of the previous
           node
7     }
8     // After the loop, current is the minimum node, and prev is its parent
9     if (prev != NULL) {
10        prev->left=NULL; // Detach the minimum node from its parent
11    }
12    // Note: This function finds and detaches, but does NOT free the node.
13    // The caller is responsible for freeing the returned node and linking it
           correctly.
14    return current; // Return the minimum node
15 }

```

↳

Functionality:

↳ Takes a pointer to the root of the subtree ('node') as input. Traverses left from the starting node to find the minimum node, keeping track of the 'prev' node (the parent of the current node). Once the minimum node is found, it detaches it from its parent by setting the parent's left pointer to 'NULL'. Returns the pointer to the detached minimum node.

↳ Note that this function only detaches the node; the caller is responsible for freeing the memory of the returned node and correctly integrating it into the deletion process (e.g., as the replacement for a node with two children).

39 Algorithmic Complexity Analysis (Binary Search Trees)

Let n be the number of log entries (nodes) in the Binary Search Tree. The complexity of BST operations depends heavily on the height of the tree, which can vary from $O(\log n)$ for a balanced tree to $O(n)$ for a skewed tree (like a linked list).

Time Complexity

Time complexity measures how the execution time of an algorithm grows with the input size.

- CreateBstLogEntry: $O(1)$ - Constant time for allocation and initialization.
- ADDNode and AddLogEntryToBst: $O(h)$ where h is the height of the tree. In the worst case (skewed tree), $h = O(n)$, so $O(n)$. In the average case (balanced tree), $h = O(\log n)$, so $O(\log n)$.
- displayBstLogEntryColumn: $O(1)$ - Constant time for printing.
- findMinBstNode: $O(h)$ - Requires traversing down the leftmost path. Worst case $O(n)$, average case $O(\log n)$.
- DelBstLogEntryNodeRecursive and DelBstLogEntryNode: $O(h)$ - Finding the node is $O(h)$, finding the in-order successor is $O(h)$, and recursive deletion of the successor is also $O(h)$. Worst case $O(n)$, average case $O(\log n)$.
- PrintBstLogEntryInOrder, PrintBstLogEntryPreOrder, PrintBstLogEntryPostOrder: $O(n)$ - Each traversal visits every node exactly once.
- CreateNodeCpyToBst: $O(1)$ - Constant time for allocation and copying.
- convertLogEntryToBst (from a linked list of size m): $O(m \times h)$ - It iterates through the linked list ($O(m)$) and for each node, inserts it into the BST ($O(h)$). Worst case $O(m \times n)$, average case $O(m \log n)$.
- Queue operations (createQueue, EnQueue, DeQueue, isEmptyQueue): $O(1)$ for a linked list based queue implementation.
- findMinBstNodeAndDel: $O(h)$ - Traversing to the minimum node. Worst case $O(n)$, average case $O(\log n)$.

Space Complexity

Space complexity measures the amount of memory used by an algorithm.

For the BST structure itself: $O(n)$ - The space required is proportional to the number of nodes. Recursive functions (ADDNode, DelBSTLogEntryNodeRecursive, traversals): $O(h)$ for the recursion stack space. Worst case $O(n)$, average case $O(\log n)$. Queue structure for traversal: $O(w)$ where w is the maximum width of the tree (number of nodes at the widest level). In the worst case (a complete binary tree), $w \approx n/2$, so $O(n)$. CreateBstLogEntry, CreateNodeCpyToBst: $O(1)$ - Constant extra space per call.

!

Summary of Binary Search Tree Operation Complexities:

Table 8: Binary Search Tree Operation Complexities

Operation	Time Complexity (Worst Case)	Space Complexity
CreateBstLogEntry	$O(1)$	$O(1)$
AddLogEntryToBst	$O(n)$	$O(n)$
DisplayBsTLogEntryColumn	$O(1)$	$O(1)$
FindMinBstNode	$O(n)$	$O(n)$
DelBsTLogEntryNode	$O(n)$	$O(n)$
In-Order Traversal	$O(n)$	$O(n)$
Pre-Order Traversal	$O(n)$	$O(n)$
Post-Order Traversal	$O(n)$	$O(n)$
CreateNodeCpyToBst	$O(1)$	$O(1)$
Convert Linked List to BST (list size m)	$O(m \times n)$	$O(n)$
Queue Operations (Enqueue, Dequeue, etc.)	$O(1)$	$O(n)$ (for queue size)
FindMinBstNodeAndDel	$O(n)$	$O(n)$

40 Conclusion

The system provides:

- Robust logging capabilities
- Efficient sorting and searching
- Comprehensive memory management
- Thread-safe operations

UX Considerations:

- Consistent prompt formatting
- Input sanitization
- Clear error messages